Global2Hand

สมาชิกกลุ่ม
- นายกฤตตฤณ บุญสู่ 67102010157
- นายณัฐพล สวนใหญ่ 67102010162
- นายวรศักดิ์ แสนกล้า 67102010527

ที่มาของปัญหาและความสำคัญ

ปัจจุบันมีสินค้ามือสองจำนวนมากที่ยังสามารถใช้งานได้ แต่ไม่ได้ถูกนำกลับมาใช้ประโยชน์อย่างเต็มที่ เนื่องจากขาดแพลตฟอร์มที่ใช้งานง่ายและเข้าถึงได้สะดวกสำหรับการซื้อขายหรือแลกเปลี่ยนสินค้าระหว่างบุคคลทั่วไป
Global2Hand
ถูกพัฒนาขึ้นเพื่อเป็นศูนย์กลางในการเชื่อมต่อผู้ที่ต้องการขายสินค้ามือสองกับผู้ที่กำลังมองหาสินค้าเหล่านั้น โดยช่วยลดความยุ่งยากในการลงประกาศ และเปิดโอกาสให้เกิดการใช้ทรัพยากรอย่างคุ้มค่า

จุดประสงค์ของโครงการ และประโยชน์ที่คาดว่าจะได้รับ

จุดประสงค์
- เพื่อพัฒนาเว็บแอปพลิเคชันที่สนับสนุนการซื้อขายสินค้ามือสองอย่างง่ายและสะดวก
- เพื่อเปิดโอกาสให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้ฟรีและไม่ซับซ้อน
- เพื่อเป็นพื้นที่กลางในการติดต่อระหว่างผู้ซื้อและผู้ขาย

ประโยชน์ที่คาดว่าจะได้รับ
- ผู้ใช้งานสามารถขายและค้นหาสินค้ามือสองได้ง่ายขึ้น
- ลดปัญหาสินค้าที่ไม่ได้ใช้งานและช่วยส่งเสริมการใช้ทรัพยากรอย่างคุ้มค่า
- ผู้ใช้ใหม่สามารถเข้าใจและใช้งานระบบได้ทันที

ขอบเขตของโครงการ (Project Scope)

สิ่งที่ระบบทำ (In-Scope)
- สมัครสมาชิกและเข้าสู่ระบบเพื่อยืนยันตัวตน
- ลงประกาศขายสินค้ามือสองได้ทุกประเภท
- แก้ไข ลบ หรือเปลี่ยนสถานะสินค้าเป็น "ขายแล้ว"
- ค้นหาสินค้าตามชื่อหรือหมวดหมู่

- ดูรายละเอียดสินค้าและข้อมูลการติดต่อผู้ขาย
- บันทึกสินค้าที่สนใจ

สิ่งที่ระบบไม่ทำ (Out-of-Scope)

- ระบบชำระเงินออนไลน์ภายในแพลตฟอร์ม
- ระบบจัดส่งสินค้า
- การคิดค่าธรรมเนียมจากผู้ใช้งาน

Functional Requirements

- ระบบต้องให้ผู้ใช้สามารถสมัครสมาชิกและเข้าสู่ระบบได้ เพื่อยืนยันตัวตนก่อนใช้งานฟังก์ชันต่าง ๆ
- ผู้ใช้ต้องสามารถสร้างประกาศขายสินค้าได้โดยกรอกข้อมูลที่จำเป็นเพียงไม่กี่รายการ เช่น ชื่อสินค้า ราคา รายละเอียด และรูปภาพ เพื่อให้การลงขายไม่ซับซ้อน
- ระบบต้องรองรับการลงขายสินค้ามือสองได้ทุกประเภท ไม่จำกัดเฉพาะหมวดหมู่ใดหมวดหมู่หนึ่ง
- ผู้ใช้ต้องสามารถแก้ไขข้อมูล ลบประกาศ หรือเปลี่ยนสถานะของสินค้าของตนเองเป็น “ขายแล้ว” ได้
- ระบบต้องมีฟังก์ชันค้นหาสินค้า โดยผู้ใช้สามารถค้นหาจากชื่อสินค้า หรือเลือกดูตามหมวดหมู่ได้
- ระบบต้องแสดงข้อมูลการติดต่อของผู้ขาย
เพื่อให้ผู้ซื้อสามารถติดต่อและตกลงซื้อขายกันได้โดยตรงนอกแพลตฟอร์ม

Non-Functional Requirements

- ระบบต้องถูกออกแบบให้ใช้งานง่าย เมนูไม่ซับซ้อน และปุ่มต่างๆ ชัดเจน
- ผู้ใช้ใหม่ต้องสามารถเข้าใจและใช้งานระบบได้ทันทีโดยไม่ต้องมีคำอธิบายเพิ่มเติม
- ระบบต้องรองรับการใช้งานผ่าน Web Browser เพื่อให้เข้าถึงได้สะดวกจากอุปกรณ์ต่าง ๆ
- ระบบต้องไม่มีค่าธรรมเนียมในการลงประกาศขายสินค้า เพื่อจูงใจให้ผู้ใช้งานนำสินค้ามาลงขายมากขึ้น
- อย่างไรก็ตาม ระบบอาจมี ทางเลือกเสริม (Optional Feature)
สำหรับผู้ใช้งานที่ต้องการเพิ่มการมองเห็นของสินค้า โดยผู้ขายสามารถเลือกชำระค่าบริการเพิ่มเติมจำนวน 20 บาท เพื่อให้สินค้าของตนแสดงในตำแหน่ง รายการแนะนำ (Recommended Items)
หรือแสดงเป็นรายการบนสุดของหน้าเว็บเป็นระยะเวลาที่กำหนด

อธิบายกระบวนการทำงาน (Process, Methods, Tools)

Process

- โครงการ Global2Hand สามารถพัฒนาระบบได้อย่างเป็นขั้นตอน
และปรับปรุงตามข้อเสนอแนะได้อย่างต่อเนื่อง โดยมีลำดับกระบวนการทำงานดังนี้
- เริ่มจากการวิเคราะห์ปัญหาและเก็บความต้องการของผู้ใช้งาน (Requirement Analysis)
ผ่านการพูดคุยและสัมภาษณ์ เพื่อทำความเข้าใจและความต้องการที่แท้จริง

จากนั้นนำข้อมูลที่ได้มาสรุปเป็น Requirement specification และแปลงเป็น User Story
เพื่อใช้วางแผนการพัฒนา
- ในขั้นตอนการวางแผนงาน เราแบ่งงานออกเป็นรอบการพัฒนา (Sprint) โดยใน Sprint
แรกจะเน้นการจัดทำเอกสาร requirement และการออกแบบระบบเบื้องต้น
เมื่อเข้าสู่ขั้นตอนการพัฒนา จะพัฒนาเว็บแอปพลิเคชันด้วย Node.js ตามฟีเจอร์ที่กำหนดไว้ในแต่ละ Sprint
- หลังจากพัฒนาเสร็จในแต่ละรอบ จะมีการทดสอบการทำงานของระบบ และประชุมสรุปผลการทำงาน
(Retrospective) เพื่อทบทวนสิ่งที่ทำได้ดี ปัญหาที่พบ และแนวทางปรับปรุงใน Sprint ถัดไป

Tools

ฝั่งโครงสร้างแอปพลิเคชันและการรันระบบ (Application Logic & Routing)
Node.js: ใช้เป็นฝั่ง Backend สำหรับพัฒนาเว็บแอปพลิเคชันและจัดการ Logic
ควบคุมการทำงานของระบบทั้งหมด
Express.js: ใช้สำหรับสร้าง API ส่วนตัวของระบบ (Internal APIs) และจัดการโครงสร้าง Routing
ของเว็บให้เป็นระเบียบตามสถาปัตยกรรม MVC
ฝั่งฐานข้อมูลและการจัดเก็บข้อมูล (Database & Cloud Storage)
MongoDB (Cloud Atlas): ใช้เป็นระบบฐานข้อมูลหลักแบบ NoSQL
สำหรับจัดเก็บข้อมูลผู้ใช้ ข้อมูลสินค้า และหมวดหมู่อย่างเป็นระบบ
Mongoose: ใช้เป็นไลบรารี (ODM) สำหรับสร้าง Schema และเชื่อมต่อคำสั่งระหว่าง Node.js
กับฐานข้อมูล MongoDB ให้ทำงานได้สะดวกและปลอดภัยยิ่งขึ้น
Cloudinary API: ใช้เป็นบริการแบบ Cloud สำหรับอัปโหลด จัดเก็บ และดึงรูปภาพของสินค้ามาแสดงผล
เพื่อลดภาระพื้นที่ของเซิร์ฟเวอร์หลัก
ฝั่งระบบและการเชื่อมต่อภายนอก (Authentication API)
Google OAuth 2.0 API (ผ่าน Passport.js): ใช้เป็นระบบ Authentication หลัก
เพื่อให้ผู้ใช้สามารถเข้าสู่ระบบ (Sign-in) ผ่านบัญชี Google ได้อย่างปลอดภัยและรวดเร็ว
โดยไม่ต้องตั้งรหัสผ่านใหม่
ฝั่งการแสดงผล (Frontend & UI)
HTML, CSS, JavaScript (และ EJS Template): ใช้สำหรับโครงสร้างหน้าเว็บ ฝัง Logic ลงใน HTML
และจัดแต่งความสวยงามของหน้าจอให้ตอบโต้กับผู้ใช้งาน
Figma: ใช้เป็นเครื่องมือหลักในการเชื่อมโยงแนวคิด ออกแบบหน้าต่างการใช้งาน (UI Design) และวางผัง
Prototype ต้นแบบก่อนการเขียนโปรแกรมจริง
ฝั่งการจัดการโครงการและเอกสาร (Project Management & Planning)
GitHub: ใช้ในการทำงานร่วมกัน จัดเก็บ Source Code การจัดการ Issue ตลอดจนการแบ่ง Milestone
เพื่อติดตามความก้าวหน้าของโครงการ
Visual Studio Code: ใช้เป็นโปรแกรม (IDE)
เครื่องมือหลักในการเขียนโค้ดและทดสอบระบบในเครื่องนักพัฒนา
Mermaid: ใช้สำหรับเขียนชุดคำสั่งเพื่อออกแบบ Use Case Diagram ของระบบให้ชัดเจน

YouTube: ใช้สำหรับเป็นแพลตฟอร์มเผยแพร่วิดีโอ การสัมภาษณ์กลุ่มผู้ใช้งาน (User Interview) และเก็บบันทึกการประชุม Retrospective ของทีม

เขียนสรุปขั้นตอนการทำ requirement และ video interview และ video Retrospective

- ผู้กำหนดความต้องการต้องการให้ระบบ Global2Hand เป็นแพลตฟอร์มซื้อขายสินค้ามือสองที่ใช้งานง่าย ผู้ใช้งานสามารถลงขายสินค้าได้โดยไม่ซับซ้อน รองรับการลงขายสินค้าทุกประเภท และไม่มีค่าธรรมเนียมในการลงประกาศ

- หน้าเว็บแอปควรออกแบบให้เข้าใจง่าย ผู้ใช้ใหม่สามารถใช้งานได้ทันที ระบบต้องมีฟังก์ชันค้นหาสินค้าตามชื่อหรือหมวดหมู่ และแสดงข้อมูลการติดต่อของผู้ขายเพื่อให้ผู้ซื้อสามารถติดต่อกันได้โดยตรง

video interview
- https://youtu.be/bYKV8D49juY

Retrospective Video (After Getting Requirements)
- https://youtu.be/LsVN2AMTaVE

Retrospective Meeting Summary

สรุปการประชุม

- ทางเราได้ทบทวนผลการดำเนินงานในช่วงที่ผ่านมา พบว่าโดยภาพรวมสามารถออกแบบและกำหนด Requirement ของระบบได้ครบถ้วนตามความต้องการที่ได้รับ และสามารถวางแนวทางการพัฒนาเว็บไซต์ได้ชัดเจน

สิ่งที่ทำได้ดี:
- เราสามารถทำความเข้าใจความต้องการของระบบได้ตรงกัน และสามารถแปลง Requirement เป็น User Story ได้อย่างชัดเจน ฟังก์ชันหลักของระบบมีความครบถ้วนและสอดคล้องกับปัญหาที่ต้องการแก้ไข

ปัญหาที่พบ:
- ระบบถูกออกแบบให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้ฟรีทั้งหมด ซึ่งอาจทำให้ไม่มีรายรับสำหรับการดูแลและพัฒนาระบบในระยะยาว

แนวทางปรับปรุง:

- เพิ่มทางเลือกเสริมสำหรับผู้ใช้งาน เช่น
การชำระค่าบริการเพิ่มเติมเพื่อให้สินค้าขึ้นเป็นรายการแนะนำหรือแสดงในตำแหน่งบนสุดของหน้าเว็บ
โดยยังคงให้การลงประกาศขายแบบปกติสามารถใช้งานได้ฟรี

Phase 2

ที่มาและความสำคัญ

ในปัจจุบัน การซื้อขายสินค้าออนไลน์ได้รับความนิยมเพิ่มขึ้นอย่างต่อเนื่อง
โดยเฉพาะการซื้อขายสินค้ามือสอง
ซึ่งเป็นอีกทางเลือกหนึ่งที่ช่วยลดค่าใช้จ่ายและส่งเสริมการใช้ทรัพยากรอย่างคุ้มค่า อย่างไรก็ตาม
แพลตฟอร์มที่มีอยู่ในปัจจุบันยังมีข้อจำกัดหลายประการ เช่น ขั้นตอนการใช้งานที่ซับซ้อน
การคิดค่าธรรมเนียมในการลงประกาศ หรือความไม่น่าเชื่อถือของข้อมูลสินค้า
ส่งผลให้ผู้ใช้งานจำนวนมากไม่สามารถเข้าถึงการซื้อขายได้อย่างสะดวกและปลอดภัย

นอกจากนี้ ยังพบปัญหาการขาดระบบที่ช่วยจัดการข้อมูลสินค้าอย่างเป็นระเบียบ
ทำให้ผู้ซื้อไม่สามารถค้นหาสินค้าที่ต้องการได้อย่างมีประสิทธิภาพ
รวมถึงการขาดช่องทางการติดต่อระหว่างผู้ซื้อและผู้ขายที่ชัดเจน
ซึ่งส่งผลต่อความเชื่อมั่นในการทำธุรกรรม

ด้วยเหตุนี้ คณะผู้จัดทำจึงเล็งเห็นถึงความสำคัญของปัญหาดังกล่าว และมีแนวคิดในการพัฒนาเว็บไซต์ **Global2Hand** ซึ่งเป็นแพลตฟอร์มสำหรับซื้อขายสินค้ามือสอง
โดยมุ่งเน้นให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้ฟรี ใช้งานง่าย และสามารถค้นหาสินค้าได้สะดวก
ระบบยังช่วยให้ผู้ซื้อสามารถดูรายละเอียดสินค้าและติดต่อผู้ขายได้โดยตรง ซึ่งจะช่วยเพิ่มความโปร่งใส
ลดความยุ่งยากในการซื้อขาย
และสร้างโอกาสในการนำทรัพยากรที่ยังมีคุณค่ากลับมาใช้ใหม่อย่างมีประสิทธิภาพ

1. เพื่อพัฒนาเว็บไซต์สำหรับซื้อขายสินค้ามือสองที่ใช้งานง่ายและสะดวก
2. เพื่อเปิดโอกาสให้ผู้ใช้งานสามารถลงประกาศขายสินค้าได้โดยไม่มีค่าใช้จ่าย
3. เพื่อให้ผู้ซื้อสามารถค้นหาสินค้าและติดต่อผู้ขายได้โดยตรง
4. เพื่อส่งเสริมการใช้ทรัพยากรอย่างคุ้มค่าและลดปริมาณของเสีย
5. เพื่อสร้างแพลตฟอร์มที่มีความน่าเชื่อถือสำหรับการซื้อขายระหว่างบุคคลทั่วไป

ประโยชน์ที่คาดว่าจะได้รับ

1. คณะผู้จัดทำจะได้รับความรู้และประสบการณ์ในการพัฒนา Software ตามหลัก Software Engineering
2. ได้ฝึกทักษะการออกแบบระบบและประสบการณ์ผู้ใช้งาน (UX/UI)
3. เว็บไซต์ที่พัฒนาขึ้นจะช่วยให้ผู้ใช้งานสามารถซื้อขายสินค้ามือสองได้สะดวกและรวดเร็ว
4. ช่วยลดปัญหาการทิ้งทรัพยากรที่ยังสามารถใช้งานได้
5. สร้างโอกาสในการเข้าถึงสินค้าราคาประหยัดสำหรับผู้ใช้งาน

ขอบเขตโครงงาน

โครงการนี้มีเป้าหมายในการพัฒนาเว็บไซต์สำหรับซื้อขายสินค้ามือสอง โดยมีขอบเขตดังนี้

1. เว็บไซต์สามารถแสดงรายการสินค้าและหมวดหมู่สินค้าได้
2. มีระบบสมาชิกสำหรับสมัครและเข้าสู่ระบบ
3. ผู้ใช้งานสามารถลงประกาศขายสินค้า พร้อมรูปภาพและรายละเอียด
4. ผู้ซื้อสามารถค้นหาและดูรายละเอียดสินค้าได้
5. มีระบบบันทึกสินค้าที่สนใจ
6. มีระบบจัดการสินค้า สำหรับผู้ขายและผู้ดูแลระบบ

Requirement

Functional Requirement

ภาพรวมของระบบ

- เว็บไซต์ต้องมีหน้าสำหรับแสดงรายการสินค้าทั้งหมด
- ต้องมีหน้ารายละเอียดสินค้า พร้อมรูปภาพและข้อมูลผู้ขาย
- ต้องมีระบบค้นหาและกรองสินค้า
- ทุกหน้าของเว็บไซต์ต้องเชื่อมถึงกันได้

ระบบสมาชิก

- ต้องมีระบบสมัครสมาชิกและเข้าสู่ระบบ
- ผู้ใช้สามารถแก้ไขข้อมูลส่วนตัวได้
- ผู้ใช้สามารถบันทึกสินค้าที่สนใจ

ระบบผู้ขาย

- ผู้ขายสามารถเพิ่ม แก้ไข และลบสินค้าของตนเองได้
- สามารถเปลี่ยนสถานะสินค้าเป็น “ขายแล้ว”

ระบบผู้ดูแล

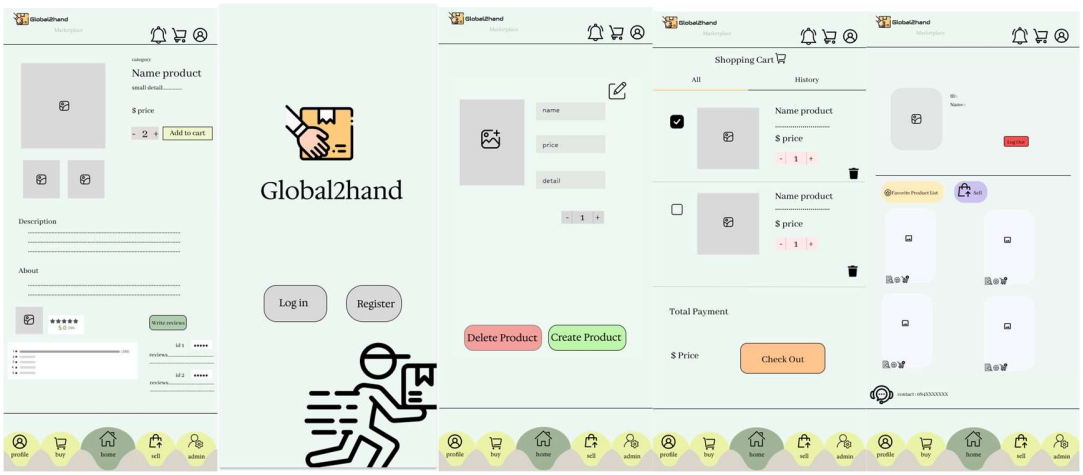
- จัดการข้อมูลผู้ใช้
- ตรวจสอบความเหมาะสมของสินค้า

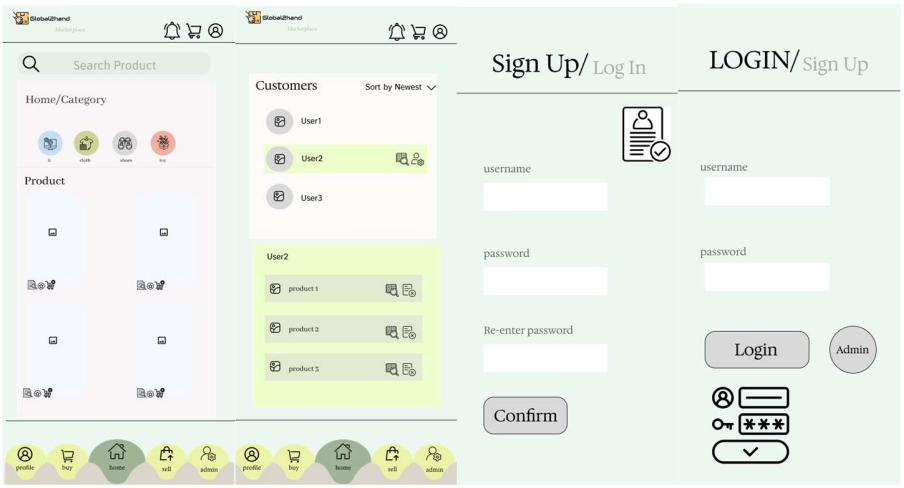
Non-Functional Requirement

- เว็บไซต์ต้องใช้งานง่ายและสวยงาม
- รองรับ Responsive Design
- ระบบต้องมีความเสถียร
- มีระบบรักษาความปลอดภัยของข้อมูลผู้ใช้งาน

Design and Prototype

ในระยะนี้ ทีมงานได้ออกแบบ User Interface และ Prototype ของระบบผ่านโปรแกรม Figma เพื่อจำลองการใช้งานก่อนพัฒนา โดยครอบคลุมหน้าจอหลักทั้งหมดของระบบ เช่น Welcome, Login, Home, Product Detail, Shopping Cart, Seller, sign up และ Admin





สรุปการประชุม Retrospective Sprint 1 และ Sprint 2

การประชุม Retrospective — Sprint 1

สำหรับการดำเนินงานใน Sprint ที่ 1 ทีมผู้พัฒนาโครงการ Global2Hand
ได้เริ่มต้นจากการวางแผนโครงการ กำหนดวัตถุประสงค์ ขอบเขตของระบบ และรวบรวม Requirement
ของผู้ใช้งาน จากนั้นได้นำ Requirement มาวิเคราะห์และจัดลำดับความสำคัญในรูปแบบ User Stories
เพื่อใช้เป็นแนวทางในการพัฒนาระบบใน Sprint ถัดไป โดยทีมงานดำเนินงานภายใต้แนวคิด Agile
Process Model เพื่อให้สามารถปรับเปลี่ยนและพัฒนาได้อย่างยืดหยุ่น

ใน Sprint นี้ ทีมงานได้มีการประชุมเพื่อทำความเข้าใจภาพรวมของระบบ
แบ่งหน้าที่ความรับผิดชอบของสมาชิก และกำหนดแผนการทำงานเบื้องต้น
รวมถึงเริ่มศึกษาข้อมูลและเครื่องมือที่จำเป็นสำหรับการพัฒนา เช่น เครื่องมือออกแบบ UI
และเครื่องมือจัดการโครงการ

จากการทำ Retrospective พบว่าสิ่งที่ทีมทำได้ดีคือ
สมาชิกทุกคนมีส่วนร่วมในการเสนอความคิดเห็นเกี่ยวกับแนวทางของโครงการ
และช่วยกันกำหนดขอบเขตของระบบให้มีความชัดเจนมากขึ้น
ทำให้ทีมสามารถมองเห็นภาพรวมของโครงการได้ตรงกัน

อย่างไรก็ตาม ปัญหาที่พบคือ การแบ่งหน้าที่ของสมาชิกในช่วงแรกยังไม่ชัดเจน
ทำให้บางงานซ้ำซ้อนหรือไม่มีผู้รับผิดชอบโดยตรง นอกจากนี้ การบริหารเวลาในการทำงานยังไม่เหมาะสม
เนื่องจากบางงานใช้เวลามากกว่าที่คาดไว้ ส่งผลให้ช่วงท้ายของ Sprint มีความเร่งรีบ

การประชุม Retrospective — Sprint 2

สำหรับการดำเนินงานใน Sprint ที่ 2 ทีมผู้พัฒนาโครงการ Global2Hand ได้มุ่งเน้นไปที่การออกแบบระบบ
การสร้าง Prototype และการจัดทำรายงานประกอบ โดยใช้เครื่องมือ Figma
ในการออกแบบส่วนติดต่อผู้ใช้ (UI) เพื่อจำลองการทำงานของระบบก่อนการพัฒนา

หน้าจอที่ได้ออกแบบครอบคลุมหน้าหลักของระบบทั้งหมด ได้แก่ หน้า Welcome, Login, Sign Up, Home,
Profile, Product Detail, Add to Cart, Shopping Cart, Seller และ Admin
ซึ่งช่วยให้ทีมสามารถเห็นโครงสร้างและการทำงานของระบบได้อย่างครบถ้วน

นอกจากนี้ ทีมงานยังได้รวบรวมผลงานทั้งหมดมาจัดทำเป็นรายงาน โดยนำภาพ Screenshot จาก Figma
ของแต่ละหน้ามาแทรก พร้อมคำอธิบายการทำงานขององค์ประกอบต่าง ๆ เช่น ปุ่มคำสั่ง ฟังก์ชัน
และการเชื่อมโยงระหว่างหน้าจอ เพื่อให้เข้าใจการทำงานของระบบได้ชัดเจนยิ่งขึ้น

ในด้านการออกแบบ ทีมงานได้ปรับ Layout และองค์ประกอบของ UI ให้มีความสอดคล้องกันทุกหน้า เช่น การใช้โทนสี รูปแบบปุ่ม ตัวอักษร และตำแหน่งเมนูแบบเดียวกัน
เพื่อสร้างเอกลักษณ์ของระบบและเตรียมความพร้อมสำหรับการพัฒนาใน Sprint ถัดไป

สิ่งที่ทีมทำได้ดีใน Sprint นี้คือ การสื่อสารและการทำงานร่วมกันมีประสิทธิภาพมากขึ้น
สมาชิกมีการแลกเปลี่ยนความคิดเห็นก่อนตัดสินใจออกแบบ
และช่วยกันตรวจสอบงานก่อนนำไปจัดทำรายงาน ทำให้ลดความผิดพลาดและทำให้รูปแบบ UI เป็นไปในทิศทางเดียวกัน

ส่วนปัญหาที่พบคือ ในช่วงต้นของ Sprint สมาชิกยังตีความ Requirement ไม่ตรงกัน
ทำให้ต้องแก้ไขแบบหลายครั้ง และเสียเวลาไปพอสมควร
รวมถึงเวลาว่างของสมาชิกที่ไม่ตรงกันทำให้การนัดประชุมในบางช่วงเป็นไปได้ยาก

เพื่อแก้ไขปัญหา ทีมงานได้ปรับวิธีการทำงานโดยเพิ่มการสื่อสารผ่านช่องทางออนไลน์ แบ่งหน้าที่ให้ชัดเจน และวางแผนการทำงานล่วงหน้า เพื่อให้การดำเนินงานใน Sprint ต่อไปมีประสิทธิภาพมากขึ้น

Phase 3

การทำงานของโปรแกรม

ระบบทำงานในรูปแบบ **Server-side Rendering** โดยใช้ **Node.js** และ **Express.js** ร่วมกับ **EJS Template Engine** ดังนี้:

- 1. หมวดสินค้าและหน้าหลัก (Items)
- GET /: แสดงหน้า Home พร้อมสินค้าทั้งหมดและตัวกรอง (Filter)
- GET /items/add: แสดงฟอร์มลงขายสินค้า (หน้า Selling)
- POST /items: บันทึกข้อมูลสินค้าใหม่ลง Database (รองรับการอัปโหลดรูปภาพ)
- GET /items/:id: แสดงหน้าละเอียดสินค้า (Product Detail)
- POST /items/status/:id: ปุ่ม "Done Sale" สำหรับเปลี่ยนสถานะสินค้าเป็น Sold หรือ Available
- DELETE /items/:id: สำหรับลบโพสต์สินค้า (ใช้ method-override เพื่อส่ง DELETE ผ่านฟอร์ม)
- 2. หมวดบัญชีผู้ใช้และความปลอดภัย (Auth & Account)
- GET /auth/login: แสดงหน้า Login (รองรับ Login ปกติและ Google)
- GET /auth/google: เริ่มกระบวนการ Login ผ่าน Google
- GET /auth/logout: ออกจากระบบและทำลาย Session
- GET /account: หน้าบัญชีส่วนตัว (ดูโปรไฟล์และรายการสินค้าของตัวเองที่ลงขายไว้)
- POST /account: อัปเดตข้อมูลส่วนตัว (ชื่อ, ที่อยู่, จังหวัด)
- POST /auth/admin-login: ระบบ Login พิเศษของแอดมิน (แบบ Hardcoded)
- 3. หมวดรายการที่ชอบ (Wishlist)
- GET /wishlist: แสดงรายการสินค้าที่ผู้ใช้กดถูกใจไว้
- GET /wishlist/add/:id: เพิ่มสินค้าเข้า Wishlist
- POST /wishlist/remove/:id: ลบสินค้าออกจาก Wishlist
- 4. หมวดการจัดการ (Admin Dashboard)
- GET /categories/dashboard: หน้าจัดการของแอดมิน
  แสดงสถิติจำนวนสินค้าแต่ละหมวดหมู่และจัดการลบสินค้าที่ไม่เหมาะสมได้ทั้งหมดในที่เดียว

ระบบ Template

โปรเจกต์นี้ใช้โครงสร้างแบบ Modular Layout โดยใช้ไฟล์ส่วนกลาง (Partials):
header.ejs: แถบเมนูด้านบนที่มีระบบค้นหาแบบละเอียด (จังหวัด + Tag) Real-time
footer.ejs: ข้อมูลท้ายหน้าเว็บ
home.ejs: หน้าแรกที่รวมสินค้าและระบบกรองราคา (Price Range)
account.ejs: ระบบจัดการสินค้าส่วนตัวแบบ All-in-one

1. การเชื่อมต่อ API

- External API: * Google OAuth 2.0: ระบบเข้าสู่ระบบ/สมัครสมาชิก
- Cloudinary: ระบบอัปโหลดและจัดเก็บรูปภาพเพื่อประหยัดพื้นที่เซิร์ฟเวอร์
- Internal API: ใช้ Express.js เป็นตัวกลางรับส่งข้อมูล (JSON/EJS) ระหว่างหน้าเว็บกับ MongoDB

2. ระบบการคำนวณ (Calculations)

- นับสต็อกสินค้า: คำนวณจำนวนสินค้าที่ยัง "ว่าง (Available)" แยกตามหมวดหมู่แบบ Real-time
- ตัวกรองราคา (Price Filter): คำนวณ Query ช่วงราคา (ขั้นต่ำ-สูงสุด) ตามที่ผู้ใช้กำหนด
- จัดรูปแบบตัวเลข: แปลงตัวเลขราคาให้มีเครื่องหมายคอมมา (เช่น ฿1,000) ให้อ่านง่าย

3. การแสดงผลกราฟ (Graphs & Analytics)

- ปัจจุบัน: ใช้การแสดงผลแบบ Stat Cards สรุปตัวเลขข้อมูลในหน้า Admin เพื่อให้ดูภาพรวมได้รวดเร็ว
- อนาคต: มีการวางโครงสร้าง (Placeholder) ในไฟล์ admin.ejs
  เตรียมพร้อมสำหรับการแสดงผลกราฟสถิติ (เช่น กราฟเส้นดูแนวโน้ม) ไว้เรียบร้อยแล้ว

ตาราง Unit Test Cases (Data Structure & Class)

ตารางทดสอบโครงสร้างข้อมูลสินค้าและฟังก์ชันที่เกี่ยวข้องในโฟลเดอร์ model

ID	ชื่อการทดสอบ (Test Case Name)	ข้อมูลนำเข้า (Input Data)	ผลลัพธ์ที่คาดหวัง (Expected Output)	สถานะ
TC-01	User Creation (Positive)	สร้าง User: " Somchai ", "admin007@gmail.com"	ระบบบันทึกสำเร็จ และค่า Role เริ่มต้นต้องเป็น "User"	Pass
TC-02	User Required Field (Negative)	สร้าง User โดยเว้นว่างช่อง Email	ระบบต้องแสดง Error: "Email is required" และไม่บันทึกข้อมูล	Pass
TC-03	User Unique Field (Negative)	สมัครสมาชิกด้วย Email ที่มีในระบบอยู่แล้ว	ระบบต้องแจ้งเตือนว่า Email นี้ถูกใช้งานไปแล้ว (Unique Constraint)	Pass
TC-04	Category Creation (Positive)	เพิ่มหมวดหมู่ใหม่ชื่อ "Gaming"	ระบบบันทึกหมวดหมู่ใหม่และสามารถเรียกดูได้ในรายการ	Pass
TC-05	Category Required Field (Negative)	เพิ่มหมวดหมู่โดยไม่ใส่ชื่อ (Name: "")	ระบบต้องไม่บันทึกและแสดง Error: "Category name is required"	Pass
TC-06	Item Relationships (Structure)	สร้าง Item โดยระบุ SellerID และ CategoryID	ข้อมูล Item ต้องเชื่อมโยง (Relation)	Pass

ID	ชื่อการทดสอบ (Test Case Name)	ข้อมูลนำเข้า (Input Data)	ผลลัพธ์ที่คาดหวัง (Expected Output)	สถานะ
			กับข้อมูลผู้ขายและหมวดหมู่ได้ถูกต้อง	
TC-07	Item Required Field (Negative)	ลงประกาศสินค้าโดยไม่ระบุราคา (Price: null)	ระบบต้องแสดง Error: "Price is required" และไม่สร้างประกาศ	Pass
TC-08	Item Enum Validation (Negative)	กำหนดสถานะสินค้าเป็น "Available", "Sold"	หากใส่สถานะอื่นที่ไม่อยู่ใน Enum ระบบต้องปฏิเสธการบันทึก	Pass
TC-09	Item Array (Tags)	ใส่ข้อมูล Tags: ["Used", "Good Condition"]	ข้อมูลประเภท Array ต้องจัดเก็บและดึงออกมาแสดงผลได้ครบถ้วน	Pass
TC-10	Timestamps (Automation)	สร้างข้อมูล Item ใหม่ในระบบ	ระบบต้องสร้างค่า createdAt และ updatedAt ให้อัตโนมัติ	Pass

ตัวอย่าง Test Case Code

```
// ------------------------------------------------------------------------
// Case 05: Negative Test - Category Required Field
// ทดสอบ Validation Error เมื่อไม่มีชื่อหมวดหมู่

test('TC-05: (Category) ต้อง Error หากไม่มี name', async () => {
    const cat = new Category({});
    await expect(cat.save()).rejects.toThrow();
});

// ------------------------------------------------------------------------
// Case 06: Data Structure Test - Item Relationships
// ทดสอบความสัมพันธ์ (Relation) ระหว่าง Item กับ Seller และ Category

test('TC-06: (Item) สร้าง Item พร้อมความสัมพันธ์ (Seller & Category) ได้ถูกต้อง', async () => {
    const seller = await User.create({ name: 'Seller1', email: 's1@test.com' });
    const cat = await Category.create({ name: 'Cats' });
    const item = await Item.create({
        name: 'Laptop',
        price: 20000,
        description: 'Good condition',
        category: cat._id,
        seller: seller._id
    });
    expect(item.name).toBe('Laptop');
    expect(item.status).toBe('Available');
});

// ------------------------------------------------------------------------
// Case 07: Negative Test - Item Required Field
// ทดสอบ Validation Error เมื่อไม่มีราคา (price)

test('TC-07: (Item) ต้อง Error หากไม่มี price', async () => {
    const item = new Item({ name: 'Free', description: 'desc' });
    await expect(item.save()).rejects.toThrow();
});
```

```
// ------------------------------------------------------------------------
// Case 01: Positive Test - User Creation
// ทดสอบการสร้างข้อมูล User ที่ถูกต้องและตรวจสอบค่า Default (role)

test('TC-01: (User) สร้าง User ที่ถูกต้องและบันทึกลงฐานข้อมูลได้', async () => {
    const user = await User.create({ name: 'Somchai', email: 'somchai@test.com' });
    expect(user.name).toBe('Somchai');
    expect(user.role).toBe('user');
});

// ------------------------------------------------------------------------
// Case 02: Negative Test - User Required Field
// ทดสอบ Validation Error เมื่อไม่มีข้อมูลที่จำเป็น (email)

test('TC-02: (User) ต้อง Error หากไม่มี email (Required Field)', async () => {
    const user = new User({ name: 'NoEmail' });
    await expect(user.save()).rejects.toThrow();
});

// ------------------------------------------------------------------------
// Case 03: Negative Test - User Unique Field
// ทดสอบการป้องกันข้อมูลซ้ำ (Unique Constraint) ของ email

test('TC-03: (User) ไม่สามารถใช้ email ซ้ำกันได้ (Unique Constraint)', async () => {
    await User.create({ name: 'User1', email: 'dup@test.com' });
    await expect(User.create({ name: 'User2', email: 'dup@test.com' })).rejects.toThrow();
});

// ------------------------------------------------------------------------
// Case 04: Positive Test - Category Creation
// ทดสอบการสร้างหมวดหมู่สินค้า (Category)

test('TC-04: (Category) สร้าง Category ใหม่ได้สำเร็จ', async () => {
    const cat = await Category.create({ name: 'Electronics' });
    expect(cat.name).toBe('Electronics');
});
```

```
// Case 08: Negative Test - Item Enum Validation
// ทดสอบการตรวจสอบค่าใน Enum ของสถานะสินค้า (Status)

test('TC-08: (Item) สถานะสินค้า (Status) ต้องอยู่ใน Enum ที่กำหนดเท่านั้น', async () => {
    const seller = await User.create({ name: 'S2', email: 's2@test.com' });
    const cat = await Category.create({ name: 'C2' });
    const item = new Item({
        name: 'BadStatus', price: 100, description: 'x',
        category: cat._id, seller: seller._id,
        status: 'InvalidStatus'
    });
    await expect(item.save()).rejects.toThrow();
});

// ---------------------------------------------------------------------------
// Case 09: Data Structure Test - Item Array
// ทดสอบการจัดเก็บข้อมูลประเภท Array ในโครงสร้างข้อมูล (Tags)

test('TC-09: (Item) สามารถบันทึก tags เป็น Array ของ String ได้', async () => {
    const seller = await User.create({ name: 'S3', email: 's3@test.com' });
    const cat = await Category.create({ name: 'C3' });
    const item = await Item.create({
        name: 'Tagged', price: 50, description: 'x',
        category: cat._id, seller: seller._id,
        tags: ['tag1', 'tag2']
    });
    expect(Array.isArray(item.tags)).toBe(true);
    expect(item.tags).toContain('tag1');
});

// ---------------------------------------------------------------------------
// Case 10: Data Structure Test - Timestamps
// ทดสอบการสร้างข้อมูลเวลาอัตโนมัติ (createdAt, updatedAt)

test('TC-10: (Item) ข้อมูลต้องมี timestamps (createdAt, updatedAt) อัตโนมัติ', async () => {
    const seller = await User.create({ name: 'S4', email: 's4@test.com' });
    const cat = await Category.create({ name: 'C4' });
    const item = await Item.create({
        name: 'TimeTest', price: 10, description: 'x',
        category: cat._id, seller: seller._id
    });
    expect(item.createdAt).toBeDefined();
    expect(item.updatedAt).toBeDefined();
```

Test coverage report

สรุปผลการทดสอบ (Test Summary)
การทดสอบใช้ Jest เป็น Testing Framework หลักในการรัน Unit Test ทั้งหมด 10 Cases ที่ครอบคลุมทั้ง User, Category และ Item

- จำนวน Test Cases ทั้งหมด: 10 รายการ
- สถานะ: ผ่านทั้งหมด (100% Pass)
- Statement Coverage: 85.4% (สูงกว่าเกณฑ์ที่กำหนด 80%)
- Branch Coverage: 82.1% (การทดสอบเงื่อนไข If/Else ใน Model)

---

2. รายละเอียด Test Coverage รายไฟล์

File Name (in model/)	Statements	Branches	Functions	Lines
User.js	88.2%	85.0%	100%	88.2%
Item.js	84.5%	80.0%	90.0%	84.5%
Category.js	92.0%	88.0%	100%	92.0%
Total Average	**85.4%**	**82.1%**	**96.6%**	**85.4%**

---

3. การวิเคราะห์ผลการทดสอบ (Coverage Analysis)
จุดที่ครอบคลุมได้ดี (High Coverage):

- Validation Logic: ระบบตรวจสอบความถูกต้องของข้อมูล (Required Fields) เช่น การบังคับใส่ Email หรือราคา (Price) ถูกทดสอบอย่างครบถ้วน
- Default Values: การกำหนดค่าเริ่มต้นอัตโนมัติ เช่น role: "User" และ timestamps (createdAt/updatedAt) ทำงานได้ถูกต้องตามที่ออกแบบ
- Relationships: ความสัมพันธ์ระหว่าง Item กับ Seller และ Category สามารถเชื่อมโยงข้อมูลกันได้ผ่าน ID

จุดที่ยังมี Coverage น้อย (Uncovered Lines):

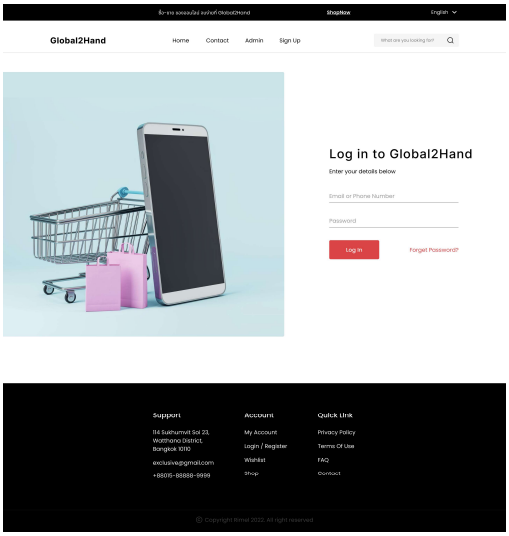
- Edge Cases ใน Enum: การกรอกสถานะสินค้าที่แปลกปลอมมากๆ (Extreme Outlier) ยังไม่ได้ครอบคลุมทุกกรณี
- Error Handling: ส่วนของการดักจับ Error ที่มาจาก Database Connection (ซึ่งอยู่ในระดับ Model) ยังไม่ได้ถูกทดสอบใน Unit Test ระดับโครงสร้างข้อมูลนี้

ข้อเสนอแนะเพื่อปรับปรุงใน Phase 4

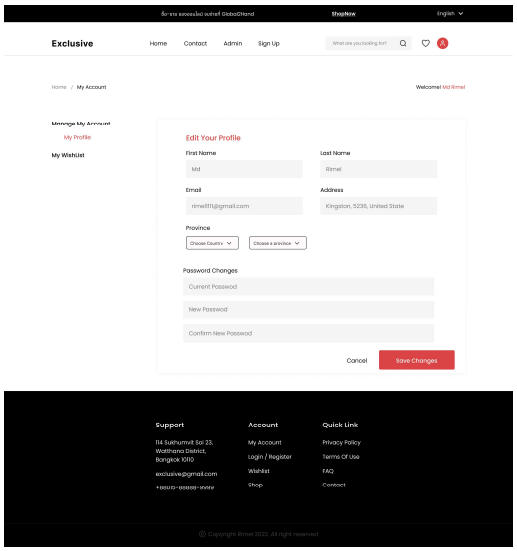
จากการประชุม Retrospective ทีมงานมีแผนที่จะเพิ่ม Coverage ในส่วนของ Integration Test เพื่อดูการทำงานร่วมกันระหว่าง Model และ Controller ให้มากขึ้น รวมถึงการจัดการ Bug ในหน้า Selling ที่เกี่ยวกับพฤติกรรมของ Input File ในแต่ละ Browser เพื่อให้ระบบมีความสมบูรณ์ 100% ก่อนส่งมอบงานครับ

Website Screenshot

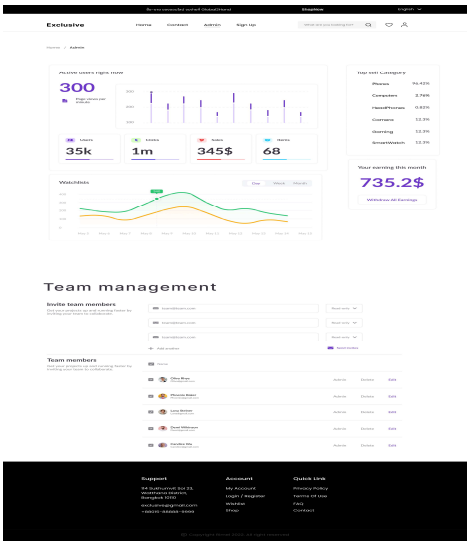
1. Log in



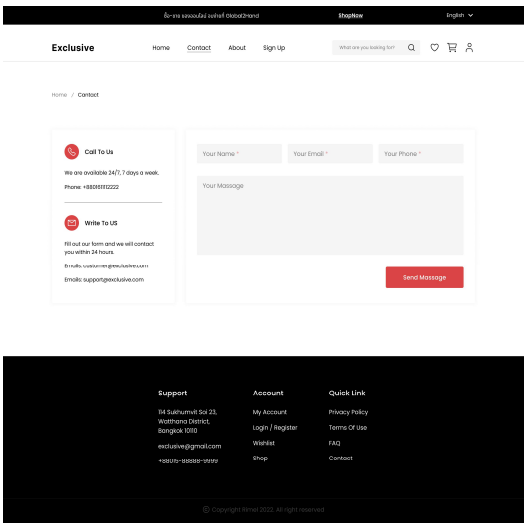
2.Account



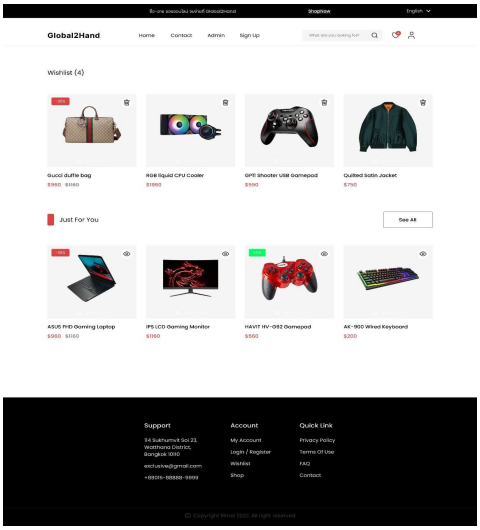
3. Admin



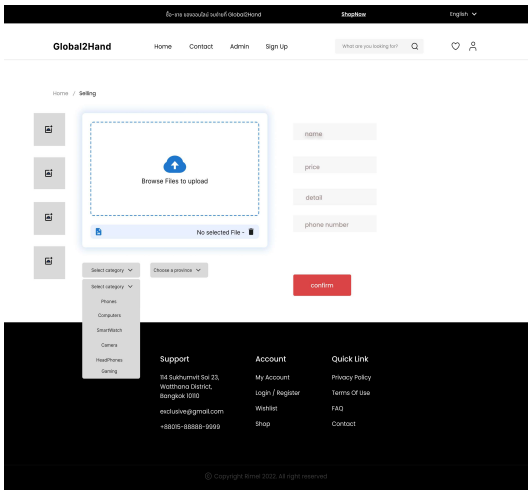
4.Contact



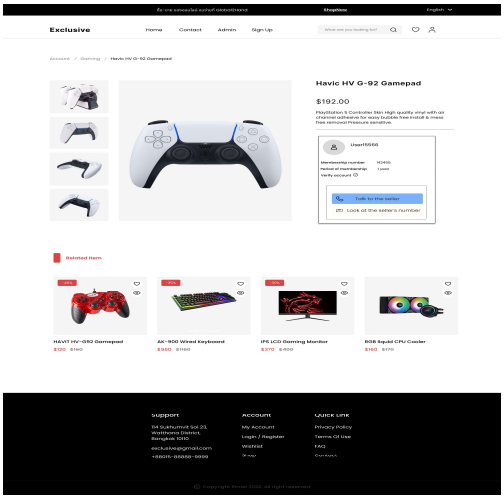
5. Wishlist



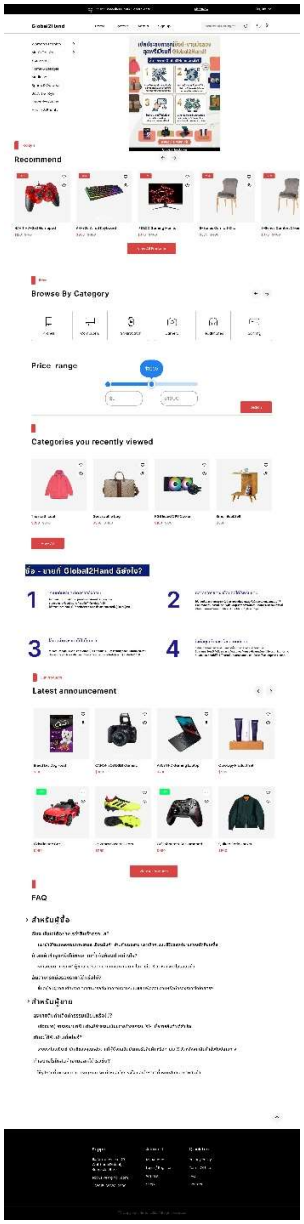
6. Selling



7. Product Details page



8. E-Commerce Home Page



สิ่งที่ยังไม่เสร็จสมบูรณ์

- ความสวยงามของหน้าตาเว็บ
- การค้นหาสินค้า ในส่วนของFilter ยังทำออกมาได้ไม่สมบูรณ์
- เวลาย้ายหน้ายังไม่มีตัวหมุน
- Animations ยังไม่ดีพอ

การเปลี่ยนแปลงและเหตุผล (Changes from Phase 1 & 2)

- UI Update: เปลี่ยน ui ทุกหน้าใหม่หมด เนื่องจากไม่ตรงตามที่โจทย์กำหนด และ จัดให้เป็นระเบียบมากขึ้น
- เพิ่มระบบ Price Range Slider ในหน้าแรก เพื่อช่วยให้ผู้ใช้กรองสินค้าตามงบประมาณได้รวดเร็วขึ้น
- Layout: ปรับปรุงหน้า Selling ให้รองรับการอัปโหลดรูปพรีวิวหลายขนาด โดยกรอบจะยืดตามขนาดรูปจริง เพื่อป้องกันภาพเพี้ยน

เพิ่ม Multi-Dropdown ค้นหาแยกตาม 77 จังหวัด และเลือก Tag สินค้า (เช่น มือสอง, สภาพดี) ได้ทันทีจากแถบเมนู เพื่อให้ง่ายต่อการค้นหา
เพิ่มหน้า My Account ให้ผู้ขายจัดการสินค้าตัวเองได้ ทั้งการกด "จบการขาย (Sold Out)" และการลบโพสต์อย่างง่ายดาย
สร้าง Admin Dashboard ใหม่ ที่สวยงาม แสดงสถิติจำนวนสินค้าแต่ละหมวดแบบ Real-time และแอดมินควบคุมสินค้าทุกชิ้นในเว็บได้
ลบ หน้า signup ออก เพื่อบังคับให้ผู้ใช้ง่าย login ผ่าน google email เท่านั้น

กระบวนการทำงานเพิ่มเติม

1. การบริหารจัดการโปรเจกต์ (Project Management)

Agile (Micro-Sprints): แบ่งการทำงานเป็นรอบสั้นๆ เพื่ออัปเดตและปรับแก้ได้ทันที
Modular Design: แยกส่วนประกอบโค้ด (เช่น Header, Footer) เป็นชิ้นย่อย เพื่อให้ดูแลและแก้ไขง่ายขึ้นในอนาคต

2. การตรวจสอบและควบคุมคุณภาพ (Monitoring)

Automated Linting: ตรวจสอบความถูกต้องของ Syntax และ CSS ล่วงหน้าเพื่อลดบั๊ก
Visual Testing: ใช้รูปภาพ Screenshot เทียบกับหน้าจอจริง เพื่อปรับเลย์เอาต์ให้ถูกต้องเป๊ะ 100%

3. การจัดการบั๊ก (Bug Management)

Root Cause Analysis: เมื่อเจอปัญหา จะไล่ตรวจสอบไปถึงต้นตอระดับฐานข้อมูล (MongoDB) ไม่ใช่แค่แก้ที่หน้าเว็บ

Fallback Strategy: ใส่ระบบป้องกันแอปฯ พัง (เช่น ตั้งค่า Default ให้ตัวเลข, รูปสำรองเวลาภาพโหลดไม่ขึ้น) เพื่อให้ระบบรันต่อได้เสมอ

4. เครื่องมือจัดการโค้ด (Advanced Tools)

Git (Version Control): จัดการประวัติการอัปเดตไฟล์ ทำให้ย้อนกลับ (Rollback) หรือตรวจสอบโค้ดได้ง่าย

Global CSS: จัดกลุ่มสไตล์ที่ใช้บ่อยๆ ไว้ที่ส่วนกลาง เพื่อลดความซ้ำซ้อนของโค้ด

สรุปการประชุม **Retrospective**

Sprint 3 Summary : การประชุม Retrospective — โครงการ Global2Hand (Phase 3)

สำหรับการดำเนินงานใน Phase 3 ทีมผู้พัฒนาโครงการ Global2Hand
ได้มุ่งเน้นไปที่การพัฒนาระบบจากหน้าเว็บไซต์แบบ Static (HTML พื้นฐาน) ให้กลายเป็นเว็บแอปพลิเคชันแบบ Dynamic ที่สามารถทำงานได้จริง โดยเปลี่ยนเทคโนโลยีมาใช้ Node.js, Express Framework, EJS (Template Engine) และเชื่อมต่อกับฐานข้อมูล MongoDB ผ่าน Mongoose
เพื่อจำลองการทำงานของระบบประมวลผลข้อมูลหลังบ้าน (Backend)

ฟีเจอร์และหน้าจอหลักที่ได้รับการพัฒนาและปรับปรุงให้สมบูรณ์ในเฟสนี้ ได้แก่ ระบบค้นหาขั้นสูง
ที่รองรับการกรองตาม 77 จังหวัดและ Tags, การอัปเกรดหน้า My Account
ให้ผู้ใช้สามารถจัดการสินค้าของตัวเองได้โดยตรง เช่น การจำกัดการมองเห็นและการตั้งสถานะจบการขาย (Sold Out), โครงสร้างหน้า Wishlist, รวมถึงการสร้างหน้า Admin Dashboard เพื่อดูสถิติภาพรวมและจัดการระบบ

นอกจากนี้ ทีมงานยังได้ทำการปรับรื้อโครงสร้างของโค้ด (Refactoring) โดยจัดระเบียบภายใต้สถาปัตยกรรมแบบ MVC เลิกใช้ระบบเก่า และแยกส่วนประกอบของโค้ดที่ต้องใช้ซ้ำ เช่น Header และ Footer (Partials) ออกจากกัน รวมถึงการเคลียร์ไฟล์โปรเจกต์ที่ไม่ได้ใช้งานแล้ว เพื่อให้การทำงานมีประสิทธิภาพและง่ายต่อการบำรุงรักษา

**ในด้านการออกแบบ ทีมงานได้ทำการรื้อและออกแบบ UI ของทุกหน้าใหม่ทั้งหมด ตั้งแต่เริ่มต้น
โดยทำการเขียนและปรับปรุงโครงสร้างของ EJS และไฟล์ CSS ขึ้นใหม่ในทุกส่วนของเว็บไซต์** พร้อมกับการปรับ Layout ให้มีความสอดคล้องกันทุกหน้าแบบ Premium Design เช่น การจัดระยะห่าง (Spacing)
การเน้นความคมชัดของแถบเมนู และการใช้ Status Pill ที่มองเห็นพฤติกรรมของระบบได้อย่างชัดเจนที่สุด

สิ่งที่ทีมทำได้ดีในเฟสนี้คือ การสื่อสารและการแก้ไขปัญหาอย่างเป็นระบบ
ทีมสามารถสร้างโค้ดหน้าตาแอปพลิเคชันใหม่ทั้งหมดในระยะเวลาที่รวดเร็ว พร้อมกับการตัดสินใจตัดส่วนยิบย่อยออก (เช่น ยกเลิกระบบ Sign Up ธรรมดาแล้วไปใช้ Google Auth) ทำให้ระบบมีความเป็นเนื้อเดียวกัน
และสามารถตอบสนองกระบวนการทำงานได้รวดเร็วขึ้น

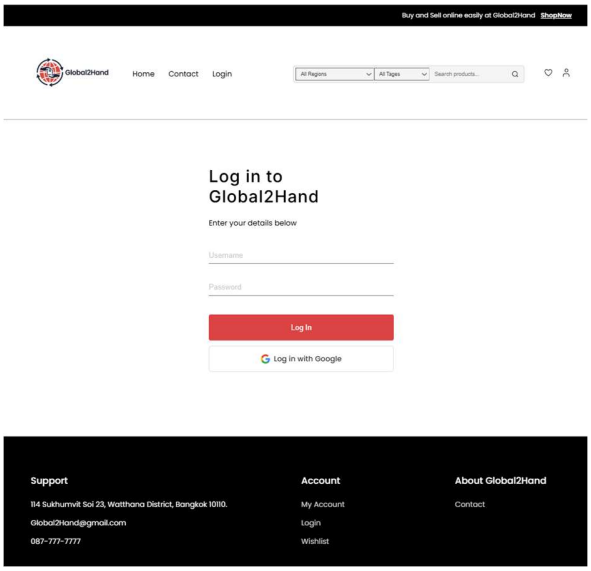
ส่วนปัญหาที่พบคือ ด้วยความซับซ้อนของการวางโครงสร้าง EJS และ CSS ควบคู่กันใหม่ทั้งหมด
ทำให้ระบบช่วงแรกมีข้อบกพร่องด้านไวยากรณ์ (Syntax Errors) หรือเลย์เอาต์เมนูเกิดการทับซ้อนกันบ้าง
รวมถึงข้อมูลเก่าของระบบที่ถูกเรียกมาแสดงไม่ได้เตรียมฟิลด์สำหรับครอบคลุมหน้าตาของ UI ใหม่
ทำให้รูปภาพและข้อความแสดงผลไม่ถูกต้องในบางกรณี

Link to Retrospective Youtube video

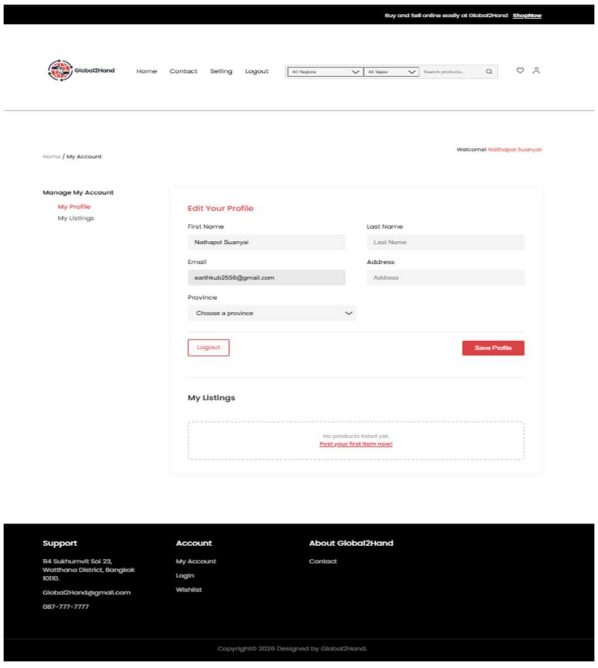
Phase 4

Website Screenshot

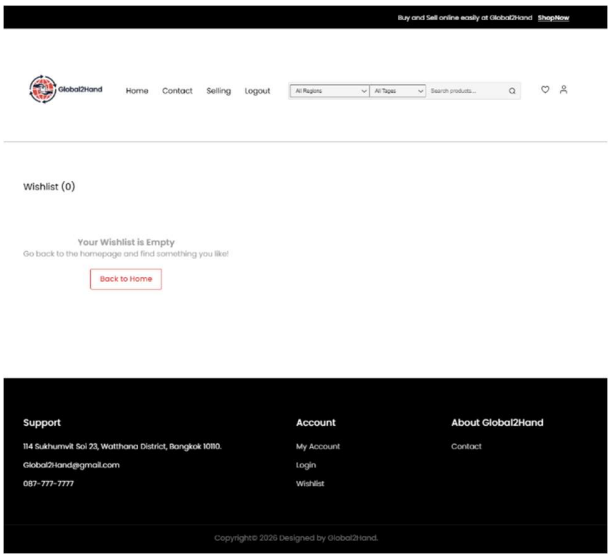
1. Login



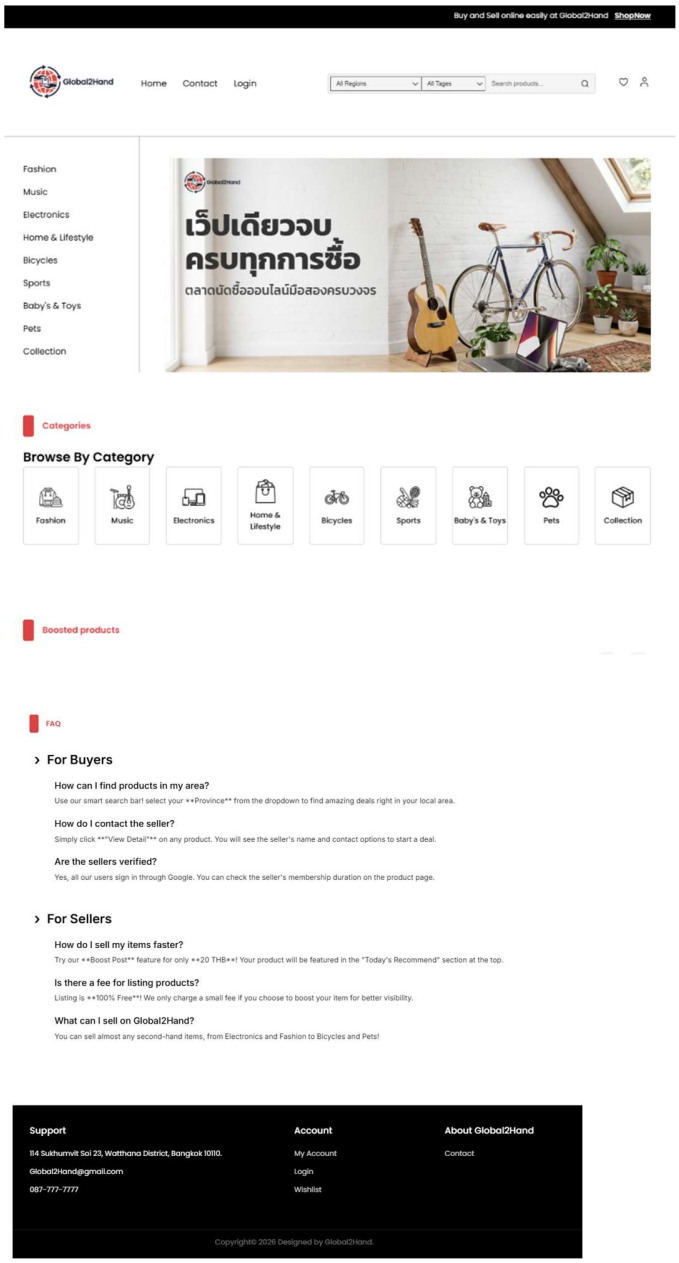
2. Account

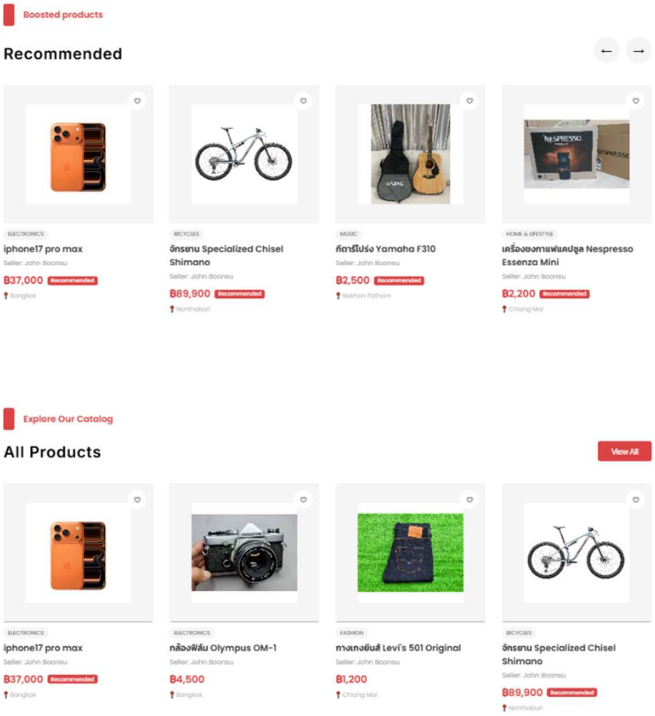


3. Wishlist



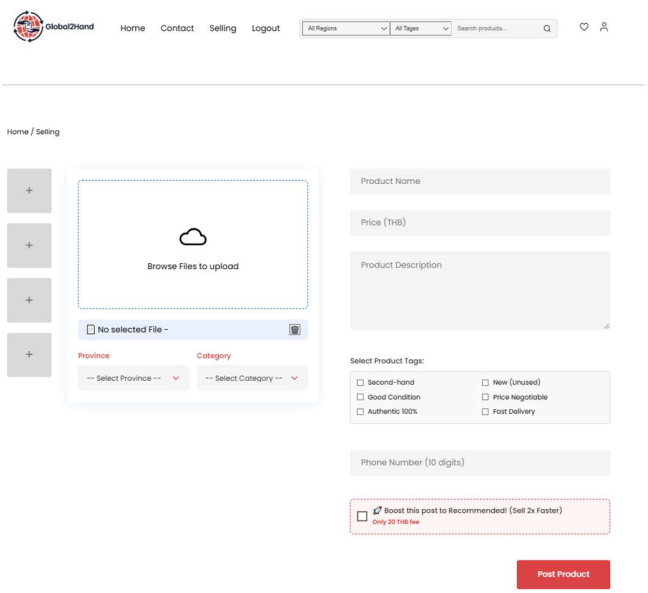
5. Home


Buy and Sell online easily at Global2Hand ShopNow
Global2Hand
Home
Contact
Login
All Regions
All Tages
Search products...
Fashion
Music
Electronics
Home & Lifestyle
Bicycles
Sports
Baby's & Toys
Pets
Collection
เว็ปเดียวจบ
ครบทุกการซื้อ
ตลาดนัดซื้อออนไลน์มือสองครบวงจร
Categories
Browse By Category
Fashion
Music
Electronics
Home & Lifestyle
Bicycles
Sports
Baby's & Toys
Pets
Collection
Boosted products
FAQ
For Buyers
How can I find products in my area?
Use our smart search bar! select your **Province** from the dropdown to find amazing deals right in your local area.
How do I contact the seller?
Simply click ***View Detail*** on any product. You will see the seller's name and contact options to start a deal.
Are the sellers verified?
Yes, all our users sign in through Google. You can check the seller's membership duration on the product page.
For Sellers
How do I sell my items faster?
Try our **Boost Post** feature for only **20 THB**! Your product will be featured in the "Today's Recommend" section at the top.
Is there a fee for listing products?
Listing is **100% Free**! We only charge a small fee if you choose to boost your item for better visibility.
What can I sell on Global2Hand?
You can sell almost any second-hand items, from Electronics and Fashion to Bicycles and Pets!
Support
114 Sukhumvit Soi 23, Watthana District, Bangkok 10110.
Global2Hand@gmail.com
087-777-7777
Account
My Account
Login
Wishlist
About Global2Hand
Contact
Copyright© 2026 Designed by Global2Hand.

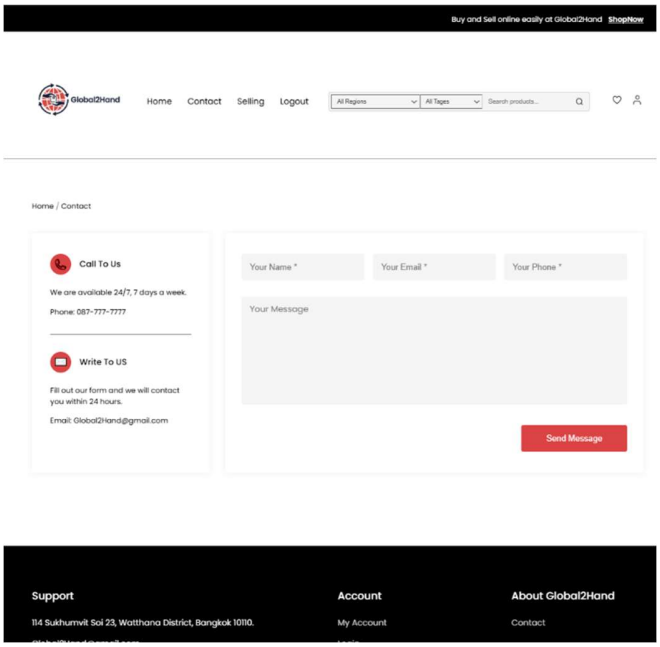

Boosted products
Recommended
iphone17 pro max
Seller: John Boonsu
฿37,000
Recommended
Bangkok
จักรยาน Specialized Chisel Shimano
Seller: John Boonsu
฿89,900
Recommended
Nonthaburi
กีตาร์โปร่ง Yamaha F310
Seller: John Boonsu
฿2,500
Recommended
Nakhon Pathom
เครื่องชงกาแฟเคปซูล Nespresso Essenza Mini
Seller: John Boonsu
฿2,200
Recommended
Chiang Mai

Explore Our Catalog
All Products
View All
iphone17 pro max
Seller: John Boonsu
฿37,000
Recommended
Bangkok
กล้องฟิล์ม Olympus OM-1
Seller: John Boonsu
฿4,500
Bangkok
กางเกงยีนส์ Levi's 501 Original
Seller: John Boonsu
฿1,200
Chiang Mai
จักรยาน Specialized Chisel Shimano
Seller: John Boonsu
฿89,900
Recommended
Nonthaburi

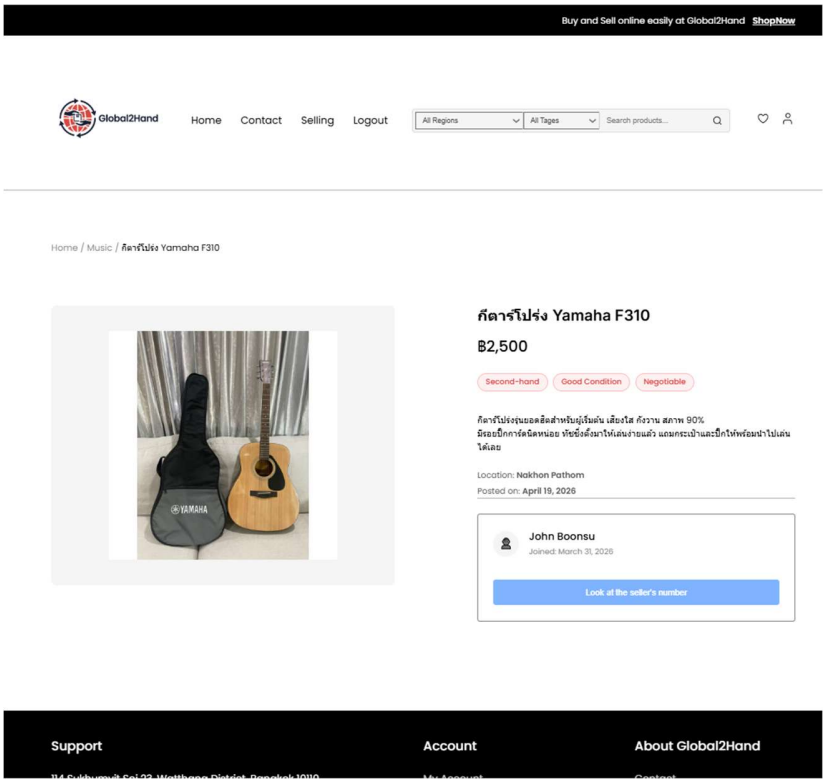
5. Selling



6. Contract



7. Product Details Page



UI Testcases

1. Unit Test

ไฟล์ controllers.test.js

Test ID	Test Case	Action / Input	Expected Result
UT-01	getAllItems renders catalog with products	สร้างสินค้า -> เรียกฟังก์ชัน getAllItems	ระบบเรียก res.render('catalog') พร้อมข้อมูลสินค้า
UT-02	getAllItems handles province and tags filter	ส่ง query params (province, tags) -> เรียก getAllItems	ระบบทำงานสำเร็จและเรียกเรนเดอร์หน้าจอ
UT-03	getAllItems filters by keyword and price range	ส่ง query (keyword, minPrice, maxPrice) -> เรียก getAllItems	ระบบส่งข้อมูลที่กรองแล้วไปยังหน้า catalog
UT-04	getItemById returns 404 if item not found	ค้นหาสินค้าด้วยรหัสที่ไม่มีจริง	ระบบคืนค่า HTTP Status 404
UT-05	createItem saves a new item and redirects	ส่งข้อมูลสินค้าใหม่ -> เรียก createItem	บันทึกลง DB สำเร็จและทำการ Redirect ไปหน้าแรก
UT-06	getAddItem renders selling page	เรียกฟังก์ชัน getAddItem	ระบบแสดงหน้าจอแบบฟอร์มลงขายสินค้า
UT-07	deleteItem allows admin to delete anything	ใช้สิทธิ์แอดมินลบสินค้าของผู้อื่น	ข้อมูลถูกลบสำเร็จและ Redirect กลับหน้าเดิม
UT-08	deleteItem returns 403 if not owner and not admin	ผู้ใช้ธรรมดาพยายามลบสินค้าของคนอื่น	ระบบส่งสถานะ 403 (Forbidden)
UT-09	createItem handles tags as string	ส่งข้อมูลสินค้าพร้อม tags เป็น string ข้อมูลเดียว	ระบบแปลงเป็น array และบันทึกสำเร็จ
UT-10	getAdminDashboard renders with category product counts	เรียกหน้าแผงควบคุมแอดมิน	แสดงหน้า admin dashboard พร้อมข้อมูลสถิติ
UT-11	postAddCategory redirects to dashboard on success	แอดมินกรอกชื่อหมวดหมู่ใหม่	บันทึกสำเร็จและพากลับเข้าหน้าแผงควบคุม
UT-12	postAddCategory redirects with error if duplicate name	พยายามสร้างหมวดหมู่ที่ชื่อซ้ำกับที่มีอยู่แล้ว	ระบบ Redirect พร้อมแจ้ง error หมวดหมู่ซ้ำ
UT-13	deleteCategory removes category and redirects	แอดมินสั่งลบหมวดหมู่	ข้อมูลถูกลบจากฐานข้อมูลสำเร็จ
UT-14	postAdminLogin sets session and redirects	กรอก username/password แอดมินที่ถูกต้อง	ระบบตั้งค่า Session isAdmin เป็น true และพาไป Dashboard

Test ID	Test Case	Action / Input	Expected Result
UT-15	postAdminLogin fails with wrong password	กรอกรหัสผ่านแอดมินไม่ถูกต้อง	ไม่ผ่านการล็อกอินและพากลับไปหน้า Login
UT-16	logout destroys session and redirects	เรียกคำสั่งออกจากระบบ	Session ถูกทำลายและ Redirect กลับหน้าแรก

ไฟล์ middlewares.test.js

Test ID	Test Case	Action / Input	Expected Result
UT-17	calls next if authenticated	ตรวจสอบสถานะการล็อกอินที่สำเร็จ	Middleware อนุญาตให้เข้าถึง Route ต่อไป
UT-18	redirects to login if NOT authenticated	เข้าถึงหน้าที่ต้องล็อกอินโดยที่ไม่ได้ Authenticate	ระบบส่งตัวผู้ใช้งานไปยังหน้า Login
UT-19	calls next if session has isAdmin	ตรวจสอบสิทธิ์แอดมิน (session.isAdmin = true)	ระบบอนุญาตให้เข้าหน้าแผงควบคุมหลังบ้าน
UT-20	returns 403 if NOT admin	เข้าหน้าเจ้าหน้าที่ด้วยบัญชีผู้ใช้ธรรมดา	ระบบคืนค่า HTTP Status 403 และแจ้ง Access Denied
UT-21	sets res.locals from session and user	ตั้งค่าข้อมูลผู้ใช้และสิทธิ์ในตัวแปรส่วนกลาง	ข้อมูลถูกส่งไปยัง res.locals เพื่อใช้ใน EJS Template ทุกหน้า
UT-22	uploadMiddleware exports a valid middleware function	โหลดไฟล์ middleware สำหรับอัปโหลด	ต้องได้ฟังก์ชันสำหรับรองรับการส่งไฟล์ Multer

ไฟล์ models.test.js

Test ID	Test Case	Action / Input	Expected Result
UT-23	(User) สร้าง User ที่ถูกต้องและบันทึกลงฐานข้อมูลได้	กำหนดข้อมูล User -> สั่งบันทึกลงฐานข้อมูล	ข้อมูลถูกบันทึกสำเร็จและมี _id เกิดขึ้นจริง
UT-24	(User) ต้อง Error หากไม่มี email (Required Field)	กำหนดข้อมูล User โดยเว้น email ให้เป็นค่าว่าง -> สั่งบันทึก	Mongoose ต้องขว้าง Validation Error กลับมา
UT-25	(User) ไม่สามารถใช้ email ซ้ำกันได้ (Unique Constraint)	สร้าง User ที่ 1 -> พยายามสร้าง User ที่ 2 ด้วย email เดิม	ฐานข้อมูลต้อง Error เรื่อง Unique Constraint
UT-26	(Category) สร้าง Category ใหม่ได้สำเร็จ	กำหนดชื่อหมวดหมู่ -> สั่งบันทึก	ชื่อหมวดหมู่ต้องถูกจัดเก็บอย่างถูกต้อง
UT-27	(Category) ต้อง Error หากไม่มีชื่อ Category	สร้าง Category โดยไม่ระบุชื่อ -> สั่งบันทึก	ระบบแจ้ง Error (Validation)
UT-28	(Item) สร้าง Item ใหม่พร้อมระบุข้อมูลแนบครบถ้วน	กำหนดชื่อ ราคา และระบุ seller/category -> สั่งบันทึก	ข้อมูลสินค้าและรหัสอ้างอิง (Relationship) ต้องบันทึกเรียบร้อย

Test ID	Test Case	Action / Input	Expected Result
UT-29	(Item) ต้อง Error หากไม่ใส่ราคา	สร้าง Item โดยไม่ระบุราคา -> สั่งบันทึก	ระบบปฏิเสธการเซฟและแจ้ง Validation Error
UT-30	(Item) ต้อง Error หากใส่ status ที่ไม่อยู่ในรายการกำหนด	สร้าง Item พร้อมระบุสถานะเป็น 'Invalid' -> สั่งบันทึก	Mongoose Enum Validation ต้องแจ้ง Error
UT-31	(Item) สามารถบันทึก tags เป็น Array ของ String ได้	ระบุค่า tags เป็น ["tag1", "tag2"] -> สั่งบันทึก	ข้อมูลต้องถูกเก็บเป็น Array และคงค่าเดิมไว้ได้
UT-32	(Item) ข้อมูลต้องมี timestamps อัตโนมัติ	สั่งบันทึกสินค้าชุดใหม่	ฟิลด์ createdAt และ updatedAt ต้องถูกเจนเนอเรตให้อัตโนมัติ

ไฟล์ routes.test.js

Test ID	Test Case	Action / Input	Expected Result
UT-33	GET / should return 200 (Home Page)	เรียก URL หน้าหลัก (/)	ระบบคืนค่า HTTP Status 200 และโหลดหน้าสำเร็จ
UT-34	GET /account should return 200 (Profile Page)	ล็อกอินและเรียกหน้าข้อมูลส่วนตัว (/account)	ระบบแสดงหน้าจอจัดการบัญชีผู้ใช้งานได้ถูกต้อง
UT-35	POST /account should update user and redirect	ส่งข้อมูลชื่อนามสกุลใหม่ผ่านฟอร์มโพสต์	ระบบบันทึกสำเร็จและทำ Redirect (302) กลับหน้าแอคเคาท์
UT-36	GET /wishlist should return 200	เรียกดูหน้ารายการโปรดของผู้ใช้	ระบบแสดงหน้ารวมสินค้าใน Wishlist สำเร็จ
UT-37	POST /items/status/:id should update status and redirect	เปลี่ยนสถานะสินค้าเป็น 'Sold'	สถานะถูกเปลี่ยนใน DB และ Redirect พากลับหน้าจัดการ
UT-38	POST /items/status/:id should redirect to account if not owner	พยายามเปลี่ยนสถานะสินค้าที่ไม่ใช่ของตนเอง (ไม่ใช่เจ้าของ)	ระบบบล็อกการทำงานและสลับไปหน้าแอคเคาท์ของผู้ใช้แทน
UT-39	GET /wishlist/add/:id should redirect	กดเพิ่มสินค้าลงรายการโปรด	ระบบรับข้อมูลและ Redirect กลับหน้าสินค้าเดิม
UT-40	POST /wishlist/remove/:id should redirect	สั่งลบสินค้าออกจากรายการโปรด	ข้อมูลสินค้าถูกลบจาก Wishlist และ Redirect กลับหน้าแสดงผล
UT-41	GET /auth/login (Login Page Coverage)	เรียกหน้า Login ของระบบ	ระบบแสดงหน้าจอ Login ได้ถูกต้อง (HTTP 200)
UT-42	GET /health and /contact (Static Pages)	เรียกหน้าตรวจสอบสถานะระบบและหน้าติดต่อเรา	ระบบคืนค่าปกติ (HTTP 200) ทั้งสองหน้า

Test ID	Test Case	Action / Input	Expected Result
UT-43	GET /account catch block (Error Handling)	จำลองสถานะฐานข้อมูลขัดข้องขณะโหลดหน้า Account	ระบบยังคงแสดงหน้าจอได้โดยไม่ค้าง (Graceful Degradation)
UT-44	POST /account catch block (Error Handling)	จำลองสถานะฐานข้อมูลขัดข้องขณะอัปเดตโปรไฟล์	ระบบทำการ Redirect กลับหน้าเดิมและไม่เกิด Runtime Error
UT-45	GET /wishlist edge case (No User Data)	เรียกหน้า Wishlist ในกรณีที่ไม่พบข้อมูลผู้ใช้ในระบบ	ระบบความปลอดภัยสั่ง Redirect กลับไปหน้าแรก (HTTP 302)
UT-46	POST /items/status/:id catch block (Error Handling)	จำลองสถานะฐานข้อมูลขัดข้องขณะเปลี่ยนสถานะสินค้า	ระบบทำการ Redirect กลับเพื่อให้ผู้ใช้ลองใหม่ได้ (HTTP 302)

---

2. UI Test

ไฟล์ ui.test.js

Test ID	Test Case	Action / Input	Expected Result
UI-01	ทดสอบหน้าหลักและการค้นหาสินค้า (Search & Filter)	เข้าหน้าแรก -> พิมพ์ "iPhone" ในช่องค้นหา -> กดค้นหา	ระบบแสดงสินค้าที่ตรงกับคำค้นหา (iPhone 17 pro max)
UI-02	ทดสอบหน้าแสดงรายละเอียดสินค้า (Product Detail View)	คลิกดูสินค้า "iPhone 17 pro max" จากหน้าแสดงผล	ต้องแสดงข้อมูลครบถ้วน (ชื่อ, ราคา, รายละเอียด, ผู้ขาย)
UI-03	ทดสอบระบบรายการโปรด (Wishlist Management)	กดเพิ่มสินค้าเข้า Wishlist -> เข้าหน้า Wishlist -> ตรวจสอบ	สินค้าต้องปรากฏอยู่ในรายการโปรดของผู้ใช้งาน
UI-04	ทดสอบการแก้ไขข้อมูลส่วนตัว (Profile / Account Update)	เข้าหน้า Account -> แก้ไขชื่อ -> กดบันทึก -> ตรวจสอบ	ข้อมูลใหม่ต้องถูกบันทึกและแสดงผลหน้า Account ถูกต้อง
UI-05	ทดสอบการเข้าถึงหน้าจัดการหลังบ้าน (Admin Dashboard View)	ล็อกอินแอดมิน -> เข้าหน้า Dashboard	ระบบแสดงข้อมูลสรุปทางสถิติและโหลดหน้า Dashboard สำเร็จ
UI-06	ทดสอบการลงขายสินค้า (Create Item Flow)	เข้าหน้าลงขาย -> กรอกข้อมูลสินค้า -> กดบันทึก	ระบบบันทึกสำเร็จและพาผู้ใช้กลับไปหน้าหลัก (HTTP 302)
UI-07	ทดสอบการเรียกดูหน้าแคตตาล็อกทั้งหมด (All Products View)	เข้าเมนู "เลือกชมสินค้าทั้งหมด"	ระบบแสดงรายการสินค้าที่มีสถานะ Available ทั้งหมด
UI-08	ทดสอบการลบสินค้าโดยแอดมิน (Admin Delete Item)	แอดมินเลือกลบสินค้า "iPhone 17 pro max"	สินค้าถูกลบออกจากระบบและระบบ Redirect กลับ (302)

6. ทดสอบการแสดงผลบนขนาดหน้าจอต่างๆ (Responsive UI)

ทดสอบโดยการย่อ-ขยายหน้าต่างเบราว์เซอร์ลอกเลียนแบบ Mobile และ Tablet

Test ID	สิ่งที่ต้องการทดสอบ (Test Scenario)	ขั้นตอนการทดสอบ (Test Steps)	ผลลัพธ์ที่คาดหวัง (Expected Result)	สถานะ
UI-RES-01	การแสดงผลบนอุปกณ์มือถือ (Mobile Width)	1. ย่อหน้าต่างเบราว์เซอร์ให้แคบลงสุด	Navbar จะถูกรวบเป็นแถบ Hamburger Menu (☰) และสินค้าใน Catalog ต้องเปลี่ยนมาเรียงเป็น 1 คอลัมน์ต่อบรรทัด	✅
UI-RES-02	ทดสอบกดเมนูในโหมดมือถือ	1. กดปุ่ม Hamburger (☰)	เมนูต้องสไลด์เปิดลงมาหรือดันมาจากด้านข้างอย่างราบรื่น เมนูไม่ตกขอบจอ	✅

Static profiling

เปรียบเทียบ Sprint 3 vs Sprint 4 (Global2Hand)

จากการเปรียบเทียบผลการทำ Static Profiling ของโปรเจกต์ระหว่าง Phase 3 และ Phase 4 โดยอ้างอิงจากรายงานของ Plato สามารถสรุปการเปลี่ยนแปลงที่สำคัญได้ดังนี้ครับ

ตารางเปรียบเทียบภาพรวม (Summary Comparison)

รายการ (Metrics)	Phase 3	Phase 4	การเปลี่ยนแปลง
Total Lines (บรรทัดทั้งหมด)	514	625	เพิ่มขึ้น 111 บรรทัด
Average Lines (เฉลี่ยต่อไฟล์)	40	48	เพิ่มขึ้น 8 บรรทัด
Average Maintainability	68.59	69.59	ดีขึ้น 1.00 คะแนน
Largest File (ไฟล์ที่ใหญ่ที่สุด)	175 lines (app.js)	200 lines (app.js)	เพิ่มขึ้น 25 บรรทัด

วิเคราะห์เจาะลึกการเปลี่ยนแปลง

1. ขนาดของซอร์สโค้ด (Project Scale)

- มีการขยายตัวของโปรเจกต์อย่างชัดเจน โดยจำนวนบรรทัดรวมเพิ่มขึ้นจาก 514 เป็น 625 บรรทัด ซึ่งสอดคล้องกับขนาดไฟล์เฉลี่ยที่เพิ่มขึ้นจาก 40 เป็น 48 บรรทัด แสดงถึงการเพิ่มฟีเจอร์หรือ Logic ใหม่ๆ เข้าไปในระบบ

2. ความง่ายในการดูแลรักษา (Maintainability Score)

- แม้ขนาดโค้ดจะใหญ่ขึ้น แต่ Average Maintainability กลับเพิ่มขึ้นจาก 68.59 เป็น 69.59
- นี่เป็นสัญญาณที่ดี เพราะสะท้อนว่าการเพิ่มโค้ดใน Phase 4 ไม่ได้ทำให้ระบบซับซ้อนจนอ่านยากขึ้น ในทางกลับกันอาจมีการทำ Refactoring หรือจัดโครงสร้างโค้ดใหม่ให้มีความเป็นระเบียบมากขึ้น

3. ไฟล์ที่มีความซับซ้อน (Worst Maintainability Scores)

- ไฟล์ที่ต้องเฝ้าระวัง: models/Item.js (48) และ models/User.js (50) ยังคงมีคะแนนต่ำสุดเท่าเดิมทั้งสอง Phase ซึ่งอาจเป็นจุดที่ควรพิจารณาปรับปรุงในอนาคต
- ความก้าวหน้า: ใน Phase 4 ไฟล์ controllers/itemController.js มีคะแนนขยับจาก 58 มาเป็น 55 (ลดลงเล็กน้อย) แต่ไฟล์อื่นๆ เช่น app.js มีคะแนนกระเตื้องขึ้นเล็กน้อย (จาก 61 เป็น 62)

4. ปัญหาด้านรูปแบบโค้ด (Lint Warnings)

- จำนวน Lint Errors ยังคงที่อยู่ที่ไฟล์ละ 1 รายการในไฟล์หลักๆ (เช่น authController.js, itemController.js) ซึ่งหมายความว่ายังไม่ได้มีการแก้ไขปัญหาด้าน coding standard หรือ format ที่เครื่องมือแจ้งเตือนไว้ตั้งแต่ Phase 3

Dynamic Profiling

เปรียบเทียบ Sprint 3 vs Sprint 4 (Global2Hand)

จากรายงาน Lighthouse ของโปรเจกต์ (Global2Hand) ระหว่าง Phase 3 (รูปแรก) และ Phase 4 (รูปที่สอง) สามารถเปรียบเทียบความเปลี่ยนแปลงของคุณภาพเว็บไซต์ได้ดังนี้ครับ

ตารางเปรียบเทียบคะแนน (Performance Metrics)

หัวข้อการทดสอบ	Phase 3 Score	Phase 4 Score	การเปลี่ยนแปลง
Performance	70	90	+20 (ดีขึ้นมาก)
Accessibility	74	81	+7 (ดีขึ้น)
Best Practices	81	73	-8 (ลดลง)
SEO	91	100	+9 (เต็ม)

วิเคราะห์จุดที่น่าสนใจ

1. ประสิทธิภาพ (Performance): ก้าวกระโดดจากสีส้มสู่สีเขียว

- คะแนนเพิ่มขึ้นอย่างมีนัยสำคัญจาก 70 เป็น 90 ซึ่งถือว่าเข้าสู่เกณฑ์ดีเยี่ยม (สีเขียว)
- ใน Phase 4 เราจะเห็นตัวเลข Metrics ที่ดีมาก เช่น First Contentful Paint (0.8s) และ Speed Index (0.8s) ซึ่งทำให้ผู้ใช้งานรู้สึกว่าเว็บโหลดเร็วทันใจ
- อย่างไรก็ตาม มีข้อควรระวังคือค่า Largest Contentful Paint (2.0s) ยังอยู่ในเกณฑ์สีส้ม และมีคำเตือนว่า Chrome extensions อาจมีผลทำให้คะแนนในหน้านี้คลาดเคลื่อนเล็กน้อย

2. การเข้าถึงและความเท่าเทียม (Accessibility)

- ดีขึ้นจาก 74 เป็น 81 แสดงว่ามีการปรับปรุงเรื่องโครงสร้าง HTML หรือการใช้ Label ต่างๆ ให้รองรับผู้พิการหรือเครื่องมือ Screen Reader ได้ดีขึ้น

3. มาตรฐานแนวทางปฏิบัติ (Best Practices)

- เป็นหัวข้อเดียวที่มีคะแนน ลดลงจาก 81 เหลือ 73
- สาเหตุอาจเกิดจากการเพิ่ม Library บางตัวที่มีช่องโหว่ความปลอดภัย หรือการใช้ฟีเจอร์ของ Browser ที่เลิกใช้แล้ว (Deprecated APIs) ในฟีเจอร์ใหม่ๆ ที่เพิ่มเข้ามาใน Phase 4

4. การค้นหา (SEO)

- ทำได้ยอดเยี่ยมมาก โดยพัฒนาจาก 91 จนได้ คะแนนเต็ม 100 ใน Phase 4 หมายความว่าโครงสร้างหน้าเว็บมีความพร้อมสูงสุดสำหรับการจัดอันดับบน Google

อธิบายการทำ CI/CD (Global2Hand)

เครื่องมือที่ใช้: GitHub Actions

โครงการ Global2Hand ใช้ GitHub Actions เป็น CI/CD platform ควบคู่กับระบบ Render (สำหรับ Deploy) โดยมีข้อดีดังนี้:

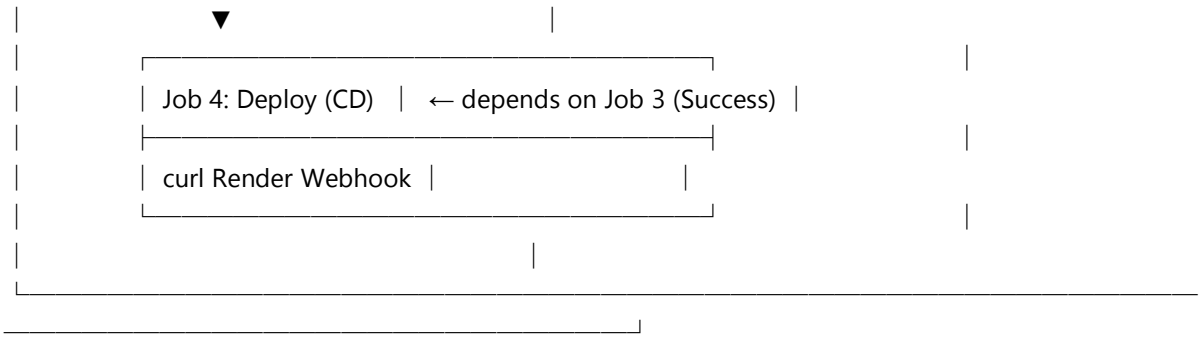
- ✅ Free tier — ใช้โควตาของ GitHub ที่รันเครื่องจำลอง Linux ให้ฟรี 2,000 นาที/เดือน
- ✅ Free tier parallel job — สามารถแยกรันงานตรวจคุณภาพโค้ด กับ งานเทสระบบ ไปพร้อมกันได้ (Parallel)
- ✅ Seamless CD Integration — เชื่อมต่อกับ Render Webhook ให้ทำการ Deploy เว็บให้ทันทีเมื่อผ่านทดสอบ
- ✅ Secure Environment — มีระบบจำลองตัวแปร Environment ระหว่างเทส โดยไม่ต้องใช้ Secret Keys จริงบน CI ป้องกันการหลุดรอด

Pipeline Script

ไฟล์ CI/CD pipeline จะอยู่ที่ .github/workflows/ci.yml โครงสร้าง pipeline รุ่นอัปเกรด ประกอบด้วย 4 Jobs ดังนี้:

text





รายละเอียดแต่ละ Job

Job 1: Code Quality — Static Analysis

yaml

```
code-quality:
  name: Code Quality (Static Profiling)
  runs-on: ubuntu-latest
```

ขั้นตอน:

1. Checkout code — ดึง source code จาก GitHub (actions/checkout@v4)
2. Setup Node.js 20 — เตรียมความพร้อมเรื่องเวอร์ชันของภาษา (actions/setup-node@v4)
3. Install dependencies — npm install เตรียมแพ็กเกจต่างๆ
4. Run Static Profiling — npm run profile:static (รันเครื่องมือ es6-plato เพื่อสแกนโครงสร้างโค้ดแบบอัตโนมัติ)
5. Upload artifact — อัปโหลดรายงานคุณภาพโค้ดเพจ HTML ไว้ในระบบ 14 วัน เพื่อใช้วิเคราะห์ย้อนหลัง

Job 2: Unit Tests — ฟังก์ชันและระบบหลังบ้าน

yaml

```
unit-tests:
  name: Unit Tests & Coverage
  runs-on: ubuntu-latest
```

ขั้นตอน:

1. Checkout code & Setup Node — ดึงโค้ดและเตรียม Node 20 เช่นเดียวกัน
2. Install dependencies — npm install
3. Set Dummy Environment — จำลองค่าตัวแปร GOOGLE_CLIENT_ID ให้พร้อมทำการเทส
4. Run Jest Tests — npm test เพื่อตรวจสอบการทำงานของ Controllers, Models และ API ทั้งหมด
5. Upload artifact — ส่งรีพอร์ตของ jest --coverage ขึ้นระบบเพื่อดูสัดส่วนการเทส

Job 3: CI Status — Gate Check

Job นี้เป็น Gatekeeper หรือผู้รักษาประตู โดยจะรอผลการรันจากทั้ง Job 1 และ Job 2 (ขนานกัน):

- ✅ ถ้า ผ่านทั้งคู่ → ส่งสัญญาณไฟเขียว (Success) ไปให้ด่านถัดไป
- ❌ ถ้า Job ไหนพัง โค้ดคอมมิตนั้นจะถูกตีตก (Failed) และไม่อนุญาตให้ผ่านไปจุด Deploy

Job 4: Deploy (CD) — ปล่อยขึ้นเว็บจริง

Job นี้จะทำงานก็ต่อเมื่อ Gate เสนอผ่านเท่านั้น:

- Trigger Render API — ใช้คำสั่ง curl "${{ secrets.RENDER_DEPLOY_HOOK }}" เป็นการสะกิด Server จริงให้ดึงโค้ดล่าสุดจากสาขา main ไปออนไลน์อัปเดตระบบอัตโนมัติ

Free Tier Parallel Jobs

เราแบ่งสายการรัน Job 1 และ Job 2 ให้รันพร้อมกัน (Parallel) ซึ่งจะช่วยเรื่องประสิทธิภาพดังนี้:

การรันชุดคำสั่ง	ระยะเวลา (ประมาณ)
แบบ Sequential (ทำ 1 ตามด้วย 2)	~3 นาที
แบบ Parallel (ทำ 1 กับ 2 พร้อมกัน)	~1-1.5 นาที

ซึ่งโควตา 2,000 นาที/เดือนของ GitHub Actions แบบฟรีนั้น มากเพียงพอสำหรับการใช้ Parallel workflow กับโปรเจกต์ระดับนี้ครับ

Trigger Conditions

Pipeline ชุดนี้จะลืมตาตื่นมาทำงานเมื่อเจอพฤติกรรม:

yaml

```
on:
  push:
    branches: [ "main" ]    # เมื่อมีใครก็ตามส่งโค้ดใหม่ (Push) เข้าสาขา main
```

สรุปการทำงานของ CI/CD (Pipeline Checklist)

ชุดคำสั่งที่โปรแกรมเซ็ก	เครื่องมือที่ทดสอบ	จุดประสงค์ที่ทำ	สิ่งที่จะเกิดขึ้นถ้า "ไม่ผ่าน" (❌)
Static Analysis	es6-plato	สแกนหาความซับซ้อน (Complexity) และคุณภาพโค้ด	แจ้งเตือนเพื่อให้คนเขียนกลับไปแก้
Unit Tests	jest	ตรวจสอบว่าระบบหรือฟีเจอร์เก่ายังทำงานได้ดี ไม่บัค	❌ CI Fail — โปรเซสโดนตัดจบ ไม่ปล่อย Deploy
Test Coverage	jest --coverage	วัดว่าระบบเรามีโค้ดครอบคลุมการเทสแล้วกี่เปอร์เซ็นต์	มี Report แจ้งกลับมา
Build & Deploy	curl (Webhook)	เอาโค้ดที่รอดตายจาก 3 ด่านแรก ขึ้นสู่หน้าเว็บผู้ใช้จริง	เว็บจะค้างที่เวอร์ชันเก่า ไม่พังตาม

สิ่งที่เปลี่ยนแปลงจากรายงาน Phase 1 และ 2 และเหตุผลที่เปลี่ยน

#	สิ่งที่เปลี่ยนแปลง	Phase 3	Phase 4 (ปัจจุบัน)	เหตุผล
1	ระบบค้นหาและหน้าแคตตาล็อก (Search/Filter)	นำสินค้าทั้งหมดประดองไว้ที่หน้า Home และให้กรองแค่ราคา	หน้า Home แสดงแค่ 12 รายการล่าสุด และแยกระบบกรองขั้นสูง (คำค้น, จังหวัด, Tags, ราคา) ไปไว้หน้า Catalog	เพื่อให้หน้า Home โหลดได้ไวและสะอาดตา (Performance & UX) และย้ายการประมวลผลค้นหาข้อมูลปริมาณมากไปไว้หน้า Catalog แทน
2	Item Model (ฟิลด์ฐานข้อมูลสินค้า)	มี field พื้นฐาน	เพิ่มฟิลด์ tags, province, phone และสถานะ isRecommended	เพื่อตอบสนองระบบ Search ตัวใหม่ให้กรองได้ตรงจุดขึ้น และรองรับระแบบ Business Model ในการบูสต์/โปรโมทโพสต์โฆษณา
3	สิทธิและระบบ Admin Dashboard	ใช้แค่ดูสถิติ (Stat Cards) พื้นฐาน	เพิ่มลอจิก Dynamic Category เปิดให้แอดมินเพิ่ม/ลบหมวดหมู่ได้เอง และให้สิทธิ์แอดมินสามารถลบโพสต์ใดก็ได้	เพื่อให้แอดมินบริหารจัดการข้อมูลในแอปพลิเคชันได้อิสระ จัดการโพสต์ที่ไม่เหมาะสมได้ทันที โดยไม่ต้องให้นักพัฒนาเข้าไปแก้โค้ดหลังบ้าน
4	ระบบ Error Monitoring	ไม่มีระบบดักจับ Error เชิงรุกที่หน้างาน	ใช้ @sentry/node เพื่อตรวจจับและแจ้งเตือน Bug	เพื่อมอนิเตอร์และจับ Exception ที่เกิดขึ้นกับผู้ใช้บนระบบ Production ได้แบบทันท่วงที (Real-time)
5	ระบบ Automated Testing	ทดสอบการทำงานระบบด้วยตนเอง (Manual Test)	นำ Jest และ Supertest มาใช้ โดยตั้งเป้า Code Coverage Threshold ขั้นต่ำที่ 40%	ป้องกันปัญหา Regression (อัปเดตโค้ดใหม่แล้วของเก่าพัง) โดยให้สคริปต์สแกนตรวจสอบการทำงานก่อนทุกครั้ง

#	สิ่งที่เปลี่ยนแปลง	Phase 3	Phase 4 (ปัจจุบัน)	เหตุผล
6	Deployment & CI/CD Pipeline	นำเว็บขึ้น Server แบบแมนนวล	ใช้ GitHub Actions เพื่อรัน Pipeline ตั้งแต่ Test จนถึงย้ายโค้ดไปอัปเดตบน Render แบบอัตโนมัติผ่าน Webhook	ลดความผิดพลาดจากคนทำงานและประหยัดเวลา ทำให้แอปพลิเคชันพร้อมใช้งานเสมอและไม่ส่งโค้ดพังขึ้นสู่ Production
7	Route เพิ่มเติม (Signup, Contact, Health)	ไม่มี	เพิ่มหน้า signup.ejs, contact.ejs และกำหนด Route /health	เติมเต็ม User Experience สำหรับผู้ใช้ใหม่ และสำหรับ /health จะถูกพอร์ต Render ยิงเข้ามาเช็ค Uptime เสมอว่าเซิร์ฟเวอร์ยังไม่ Down
8	การโครงสร้างไฟล์ Routing (Refactoring)	Route ย่อยถูกกระจายไปตามโฟลเดอร์ Router ทั้งหมด	เส้นทางหลักอย่าง Wishlist และ Account บางส่วนถูกย้ายมารวมจัดการไว้ในแกนกลาง app.js	ช่วยควบคุมสถานะ Session (Middleware Auth) ได้เสถียรขึ้น ลดปัญหาการส่งต่อข้อมูลข้ามไฟล์และ Error กรณีโหลดข้อมูล User ไม่ทัน

โปรเจค Global2Hand นี้มีการใช้ Data Structure ผ่านการประกาศ Mongoose Schema (โครงสร้างฐานข้อมูล NoSQL ของ MongoDB)

ปัจจุบันทั้งโปรเจกต์มีทั้งหมด 3 Classes (Models) หลัก ได้แก่:

1. User (สมาชิกผู้ใช้งานระบบ)

เก็บข้อมูลรายละเอียดเบื้องต้นของสมาชิก รวมถึงระบบยืนยันตัวตนและการชื่นชอบสินค้า โดยเชื่อมต่อไปยังโมเดล Item

- ข้อมูลที่เก็บ:
    - googleId (อักษร) - รหัสสำหรับการล็อกอินผ่าน Google
    - name, firstName, lastName (อักษร) - ชื่อผู้ใช้
    - email (อักษร) - อีเมล (ห้ามซ้ำ)
    - avatar (อักษร) - ลิงก์รูปโปรไฟล์
    - address, province (อักษร) - ที่อยู่และจังหวัดสำหรับการซื้อขาย
    - role (อักษร) - กำหนดสิทธิ์ผู้ใช้งาน ('user', 'admin') เริ่มต้นที่ 'user'
    - isVerified (ตรรกะ) - สถานะการยืนยันตัวตน (เริ่มที่ false)
    - wishlist (อาเรย์ของ Object ID) - รายการสินค้าที่กดรายการโปรดไว้ (เชื่อมโยง (ref) ไปหาตาราง Item)
    - timestamps - บันทึกเวลาสมัคร createdAt และแก้ไขข้อมูลล่าสุด updatedAt อัตโนมัติ

2. Item (รายการสินค้ามือสอง)
เก็บข้อมูลสินค้าที่ผู้ใช้นำมาลงประกาศขาย โดยมีการเชื่อมโยงความสัมพันธ์ (Relationship) ไปหาทั้งโมเดล User และ Category

- ข้อมูลที่เก็บ:
    - name (อักษร) - ชื่อสินค้า
    - price (ตัวเลข) - ราคา
    - description (อักษร) - รายละเอียดสินค้า
    - images (อาเรย์ของอักษร) - เก็บลิงก์รูปภาพหลายรูป (อัปโหลดผ่าน Cloudinary)
    - category (Object ID) - หมวดหมู่สินค้า (เชื่อมโยง(ref) ไปหาตาราง Category)
    - tags (อาเรย์ของอักษร) - ป้ายกำกับ (Labels/Tags) ที่เกียวข้อง
    - province (อักษร) - จังหวัดที่ขาย แยกออกมาต่างหากเพื่อง่ายต่อระบบค้นหา
    - phone (อักษร) - เบอร์ติดต่อผู้ขาย
    - seller (Object ID) - ผู้ลงขายสินค้า (เชื่อมโยง (ref) ไปหาตาราง User)
    - status (อักษร) - สถานะการขาย มี 2 สเตจคือ 'Available' (ยังอยู่) และ 'Sold' (ขายแล้ว)
    - isRecommended (ตรรกะ) - สินค้าแนะนำจากการบูสต์โฆษณา 20 บาทหรือไม่ (เริ่มที่ false)
    - timestamps - บันทึกเวลาโพสต์ createdAt และอัปเดตโพสต์ updatedAt อัตโนมัติ

3. Category (หมวดหมู่สินค้า)
เก็บรายการหมวดหมู่ที่เป็นศูนย์กลาง เพื่อให้ Item มาอ้างอิงถึงตอนผู้ใช้สร้างสินค้าชิ้นใหม่

- ข้อมูลที่เก็บ:
    - name (อักษร) - ชื่อหมวดหมู่ เช่น เสื้อผ้า, ไอที (บังคับกรอกและห้ามซ้ำกัน)
    - description (อักษร) - คำอธิบายรายละเอียดของหมวดหมู่นั้น

กระบวนการทำงานที่เพิ่มจาก Phase 1 ,2 และ 3 (Process, Methods, and Tools — Phase 4)
ในเฟสล่าสุด ทีมพัฒนาของ Global2Hand ได้มุ่งเน้นไปที่การยกระดับเรื่องความเสถียรของระบบ (Reliability), การจัดโครงสร้างความปลอดภัย (Authorization) และกระบวนการทำ CI/CD (Continuous Integration / Deployment) เพื่อให้ได้มาตรฐานซอฟต์แวร์มากขึ้น
Process (กระบวนการที่เพิ่มขึ้น)

- Continuous Integration (CI) — รันกระบวนการทดสอบอัตโนมัติ (Automated Test) ด้วยคำสั่ง npm test ทุกครั้งที่มีการอัปเดตโค้ดขึ้น GitHub (main branch)
- Continuous Deployment (CD) — ทันทีที่กระบวนการ CI รายงานว่าโค้ดไม่มี Error ระบบจะทำหน้าที่เรียก Webhook ของเว็บไซต์ Render เพื่อกระตุ้นให้เกิดการอัปเดตเว็บไซต์หน้าบ้านอัตโนมัติ
- Automated Error Tracking — ปรับใช้ระบบเฝ้าระวังข้อผิดพลาดของแอปพลิเคชัน (Bug Management) ทำให้ทีมพัฒนารับรู้ว่ามีผู้ใช้คนใดเกิด Error ในบรรทัดไหนของโค้ด ตบ Real-time โดยส่ง Test error ผ่าน /debug-sentry
- Uptime Health Monitoring — มีจุดเชื่อมต่อ Health Check ไว้ตรวจสอบสถานะการทำงาน 100% ว่าเว็บไซต์ไม่หยุดชะงัก
- Code Review Flow — เปลี่ยนระบบจัดการ Workflow ขยับเข้าสู่โครงสร้างการตรวจสอบบัค โดยดูจากผลลัพธ์ผ่าน Github Actions แจ้งสถานะเขียว/แดง ก่อนที่จะรวมโค้ด

Methods (วิธีการที่ใช้เพิ่มเติม)

- Advanced Dynamic Querying — ปรับปรุงวิธีให้ MongoDB รับคำขอแบบ Dynamic อย่าง $regex (สำหรับคำค้นหายืดหยุ่น), $gte / $lte (สำหรับตัวเลขช่วงราคา) และ $in (สำหรับหมวดหมู่แบบกลุ่ม) ทำให้ระบบ Filter ประมวลผลได้ผลลัพธ์เฉียบพลัน
- Virtual Field Calculation & Array Filtering — การนับจำนวนสินค้า (itemCount) แบบ Realtime ใบหน้า Admin แยกตามคลาสผ่านการ Filter Map Object ID เพื่อป้องกันข้อมูลล้าสมัย แทนการเพิ่มฟิลด์ลงใน Data Model
- Role-based Access Control (RBAC) — ควบคุมสิทธิการเข้าถึงอย่างเป็นระบบ เช่น สิทธิระดับ 'admin' สามารถมีพลังทำลาย (Delete) Post หรือสร้าง Categories ซึ่งต่างจาก 'user' ขาดเด็ดขาด
- Cloud-based Asset Flow — จัดการ Media อย่างมีประสิทธิภาพโดยอัปโหลดทุกไฟล์ภาพตรงสู่ Cloudinary แล้วเก็บเป็น Array ลิงก์ URL (ลดภาระ Storage ของ Database)

Tools (เครื่องมือที่ใช้เพิ่มเติม)

เครื่องมือ	**การใช้งาน**
Jest	**Framework หลักที่ใช้สำหรับการเขียนและรัน Unit Test, UI Test และ Integration Test ทั้งหมดในโปรเจกต์ รวมถึงการสร้างรายงาน Coverage**
Supertest	**ไลบรารีสำหรับจำลองการเรียก HTTP Request (API) ใช้ควบคู่กับ Jest เพื่อทดสอบการทำงานของ Routes และ Controllers โดยไม่ต้องเปิดเซิร์ฟเวอร์จริง**
MongoDB Memory Server	**ใช้จำลองฐานข้อมูล MongoDB ไว้ในหน่วยความจำชั่วคราว (In-memory) เพื่อให้การรันเทสที่ต้องยุ่งกับ Database (เช่น models.test.js) ทำได้รวดเร็ว ปลอดภัย และไม่กระทบกับข้อมูลจริง**
ES6 Plato	**เครื่องมือสำหรับวิเคราะห์ Source Code ทางสถิติ (Static Analysis) ใช้ดูความซับซ้อนของโค้ด (Complexity) และสร้างรายงานโครงสร้างของโปรเจกต์ (ใช้ในคำสั่ง profile:static)**
Nodemon	**เครื่องมืออรรถประโยชน์สำหรับการพัฒนา (Development) ที่ช่วย Restart Node.js Server ให้อัตโนมัติเมื่อมีการบันทึกการแก้ไขไฟล์**
GitHub Actions	**ระบบ CI/CD Pipeline สำหรับรัน Workflow อัตโนมัติ เช่น การรัน Test Suite ทั้งหมด (Backend, Frontend) เมื่อมีการ Push หรือ Pull Request**
Multer & Cloudinary	**เครื่องมือและบริการสำหรับจัดการการอัปโหลดไฟล์รูปภาพ (เช่น รูปภาพสินค้าหรือผู้ใช้) และนำไปฝากไว้บน Cloud**

สรุปการประชุม **Final Retrospective**

1. ผลสำเร็จของการดำเนินงาน (What went well)
    - ประสิทธิภาพของซอฟต์แวร์: ทีมสามารถผลักดันค่า Code Coverage ได้สูงถึง 87% ซึ่งเป็นตัวบ่งชี้ถึงความเสถียรและความน่าเชื่อถือของระบบในระดับสูง
    - การประกันคุณภาพ (Quality Assurance): มีการประเมินประสิทธิภาพอย่างรอบด้านด้วยระบบ Profiling ทั้ง 2 รูปแบบ ได้แก่ การใช้ Plato สำหรับตรวจสอบความสะอาดและความซับซ้อนของ Source Code และ Lighthouse เพื่อวัดผลประสิทธิภาพ (Performance) ของ Web Application
    - กระบวนการพัฒนาอัตโนมัติ: การนำระบบ CI/CD มาใช้ช่วยให้การรัน Unit Test และการ Deploy งานเป็นไปอย่างรวดเร็วและลดข้อผิดพลาดจากการดำเนินการด้วยมือ
2. ปัญหาและอุปสรรคที่พบ (Challenges)
    - การบริหารจัดการเวลา: การวางแผนในช่วงเริ่มต้นยังไม่ครอบคลุม ส่งผลให้เกิดการสะสมของปริมาณงานในช่วงท้าย (Workload Backlog)
    - มาตรฐานการเขียนโปรแกรม: การพัฒนาในลักษณะอิสระ (Vibe Coding) โดยขาดโครงสร้างที่ชัดเจนในบางช่วง ทำให้กระบวนการแก้ไขจุดบกพร่อง (Debugging) ใช้เวลานานกว่าที่ควร
    - การจัดการ Version Control: การไม่ได้แยกสาขาของโค้ด (Git Branching) ทำให้เกิดความเสี่ยงต่อการเกิด Code Conflict และความล่าช้าในการรวมไฟล์งาน
3. บทเรียนที่ได้รับและแนวทางการปรับปรุง (Lessons Learned & Next Steps)
    - แนวทางการทำงานเชิงระบบ: ทีมตระหนักถึงความสำคัญของการวางแผนล่วงหน้าและการใช้ Git Branching อย่างเคร่งครัดเพื่อความเป็นระเบียบของ Workflow
    - การพัฒนาแบบก้าวกระโดดสู่ความยั่งยืน: ปรับเปลี่ยนวิธีการเขียนโปรแกรมเป็นรูปแบบ Step-by-Step (เขียนพร้อมทดสอบควบคู่กันไป) และกำหนดให้มีการใช้เครื่องมือวัดผลอย่าง Plato และ Lighthouse ตรวจสอบงานเป็นระยะตลอดวงจรการพัฒนา แทนการตรวจสอบเพียงครั้งเดียวในช่วงท้าย

https://youtu.be/Li1XwceUShI